Planul proiectului, feature cu feature

Spring Boot 3.5 · Java 21 · MySQL 8.4 · Flyway · Testcontainers · Spring AI 1.1 (MCP) · Kafka · Prometheus + Grafana · Tempo + ELK · Keycloak · Kong · Angular · doar Docker, pe EC2 · Sebi

Pornești de la codul tău, nu de la zero. Ai deja piesele grele: `BatteryCommandValidator` (limitele bateriei, cu randament), `EnergyEngine` (bilanțul casei), `AgentMode` (cele 4 niveluri de autonomie), `SimulationProperties`, `GridProperties`, `ApplicationIntializer` (adminul din `.env`), contul `AGENT` din `UserType` și un pipeline cu 4 joburi în Actions. Starea de azi (`f3bd97b`, rulată): **nu compilează** (F1 o repară), schema o face încă Hibernate, iar serviciul bateriei are corpurile goale (F3). Planul leagă piesele, pe rând.
Înlocuiește `PLAN_BACKEND.pdf` și `FEATURES.pdf`. Tabelele rămân în `STRUCTURA_BD.pdf`, contractul REST în `openapi.yaml`. Bug-urile din `CODE_REVIEW.pdf` (runda 8) se repară **în feature-ul căruia îi aparțin**: le găsești la „Repară” / „Ce lipsește sau e stricat”.
Primele 5 feature-uri le faci singur, cu ce ai învățat deja în Spring (Boot, JPA, teste cu Mockito și MockMvc, Security, Docker). Singura noutate din ele e **Flyway, în F1** (lecția `spring-flyway`, o termini înainte de F1); F2–F5 nu aduc nimic nou, fiecare are doar câte o migrare. De la F6 încolo, câte o unealtă nouă pe feature, fiecare cu lecția ei, marcată **nou**: F6 aduce Testcontainers + failsafe (`spring-testcontainers`). Antetul fiecărui feature spune: „Cu ce știi: da” sau „Nou: … — lecția … înainte”.
Scopul: să construiești singur tot procesul unui sistem real: microservicii, Spring AI prin MCP, logare, metrici, tracing, Kafka, un server de identitate (Keycloak), un API gateway (Kong) și interfața în Angular, fiecare intrat în proiect la momentul în care îți rezolvă o problemă. Totul pe **Docker** (docker compose pe EC2), fără Kubernetes.
Fiecare feature merge de la A la Z, în aceeași ordine:

ce poate face utilizatorul → migrare `Vn__….sql` → model + repository → service → intrare (REST, tool MCP sau Kafka) → permisiuni → test cu login real → log / metrică → verde în Actions → verificat (local; pe EC2 acolo unde „Gata când” spune „Pe EC2”)
Un feature e gata doar când trece „Gata când” cu token din `/auth/login`: local în F1–F5, iar F1 și F5 au și un rând „Pe EC2” (deploy-ul trebuie să țină din primul feature); de la F7, pe EC2. Nu începi următorul înainte.

Toate feature-urile

F	Feature	Cu ce știi / Nou + lecția înainte	Migrare
F1	Contul meu	**nou** Flyway — `spring-flyway` (ex. 1–3 făcute, 4–8 de terminat)	`V1`
F2	Casa cu bateria ei	Cu ce știi: da (a doua migrare = ex. 5 din `spring-flyway`)	`V2`
F3	Comand bateria — poarta	Cu ce știi: da	`V3`
F4	O zi simulată	Cu ce știi: da (tick la cerere, log simplu)	`V4`
F5	Banii casei	Cu ce știi: da (`BigDecimal` e Java de bază)	`V5`
F6	Testele rulează pe MySQL real	**nou** Testcontainers + failsafe — `spring-testcontainers` (gata)	–
F7	Logarea zilei simulate	**nou** logare structurată (MDC) — `spring-logare-structurata` (de făcut)	–
F8	Starea rețelei	**nou** Micrometer + Prometheus + Grafana — `devops/metrici-prometheus-grafana` (de făcut)	`V6`
F9	Comunitatea văzută de un agent	**nou** server MCP — `java/spring-ai/mcp-server` (de făcut)	–
F10	Propun → aprob → execut	**nou** tool-uri MCP de scriere — `mcp-server`, partea a doua (de făcut)	`V7`
F11	Brokerul pe reguli — primul microserviciu	**nou** client MCP — `java/spring-ai/mcp-client` (de făcut)	–
F12	Brokerul cu AI	**nou** Spring AI `ChatClient` — `java/spring-ai/chat-client` (de făcut)	–
F13	Tick-ul anunță, ceilalți reacționează	**nou** Kafka — `java/spring/spring-kafka` (de făcut)	–
F14	Simulatorul pleacă din monolit	**nou** topicurile ca contract — `java/microservicii/al-doilea-serviciu` (de făcut)	–
F15	Urmăresc un tick prin tot sistemul	**nou** OpenTelemetry + Tempo + ELK — `devops/tracing-otel-tempo-elk` (de făcut)	–
F16	Cine ești, spus de Keycloak	**nou** Keycloak — `java/spring/spring-security-keycloak` (de făcut)	`V8`
F17	O singură poartă: Kong	**nou** Kong + rețele Docker — `devops/kong-gateway` (de făcut)	–

Lecțiile `spring-flyway` și `spring-testcontainers` sunt scrise pe Spring Boot 4.1; proiectul tău e pe Boot 3.5 (nu se schimbă până la susținere). Unde diferă (starter-ul Flyway, artefactele și importurile Testcontainers) e spus la pasul respectiv din F1 și F6.

Unde ajung fostele F1–F15

Ai citit planul vechi; aceeași muncă, alte numere. U1–U4 își păstrează numele; se schimbă doar „după Fx”.

Vechi	Nou	Ce s-a mutat
F1 Contul meu	**F1** + **F6**	Flyway rămâne în F1, cu `V1` = schema de la finalul F1. Testcontainers, failsafe, CI pe PR și B9 se mută în F6, ca feature separat. B2 și compilarea rămân în F1.
F2 Casa cu bateria ei	**F2**	La fel (`V2`, a doua migrare); fără nimic nou.
F3 Poarta	**F3**	La fel (`V3`); fără nimic nou.
F4 O zi simulată	**F4** + **F7**	Logarea structurată (`tickId` în MDC, JSON pe EC2) iese din F4 și devine F7. F4 rămâne cu `log.info` simplu.
F5 Banii casei	**F5**	La fel; `BigDecimal` nu mai e „nou”.
F6 Starea rețelei	**F8**	`V6`, Micrometer + Prometheus + Grafana.
F7 Comunitatea văzută de un agent	**F9**	Server MCP.
F8 Propun → aprob → execut	**F10**	`V7`; aprobare și respingere.
F9 Brokerul pe reguli	**F11**	Primul microserviciu.
F10 Brokerul cu AI	**F12**	Spring AI.
F11 Tick-ul anunță	**F13**	Kafka.
F12 Simulatorul pleacă	**F14**	Al doilea serviciu.
F13 Urmăresc un tick	**F15**	OpenTelemetry + ELK.
F14 Keycloak	**F16**	`V8`.
F15 Kong	**F17**	Poarta.
U1 după F3 · U2 după F6 · U3 după F8 · U4 între F14 și F15	U1 după F3 · U2 după **F8** · U3 după **F10** · U4 între **F16** și **F17**	Interfața urmează endpoint-urile, nu lecțiile de backend.

Unde ajungi

```
Internet → nginx (TLS) → ui Angular (U1–U4) · Kong (F17) → solar-sync-broker-api ── MySQL
                                                          │ MCP (F9–F10)
                                                          └ solar-sync-broker-agent (F11–F12) → model AI
                         └ oauth2-proxy → Grafana · Kibana · Kafka UI
simulator (F14) → Kafka (F13) → api → Kafka → agent
Keycloak (F16) emite tokenurile · Prometheus + Grafana (F8)
Tempo + Elasticsearch / Kibana / Filebeat (F15) peste toate
```

Diagrama completă, cu rețelele Docker și profilurile compose: `ARHITECTURA.pdf`, lângă acest plan.

Lecții de predat, în ordine

Înainte de	Lecția	Noțiuni	Starea
F1	`java/spring/spring-flyway`	migrări versionate (`V1__…`), `validate` ; ex. 4–8: câmp nou fără migrare, a doua migrare, baseline, migrare care pică	gata; ex. 1–3 făcute, 4–8 de făcut
F2 · F3 · F4 · F5	nicio lecție: le faci singur (F2 după ex. 5 din `spring-flyway`)		
F6	`java/spring/spring-testcontainers`	MySQL real în teste, `*IT` + failsafe, `mvn verify` , CI pe PR	gata, verificată; nelivrată
F7	`java/spring/spring-logare-structurata`	MDC, loguri JSON, niveluri	de făcut
F8	`devops/metrici-prometheus-grafana`	Micrometer, `/actuator/prometheus` , Prometheus, Grafana	de făcut
F9	`java/spring-ai/mcp-server`	server MCP, tool-uri de citire, JWT	de făcut
F10	`java/spring-ai/mcp-server` , partea a doua	tool-uri de scriere = propunere → aprobare	de făcut
F11	`java/spring-ai/mcp-client`	client MCP, a doua aplicație, `@Scheduled` pentru buclă	de făcut
F12	`java/spring-ai/chat-client`	`ChatClient` , ieșire structurată, fallback	de făcut
F13	`java/spring/spring-kafka`	producători, `@KafkaListener` , idempotență, Kafka UI	de făcut
F14	`java/microservicii/al-doilea-serviciu`	topicurile ca singur contract	de făcut
F15	`devops/tracing-otel-tempo-elk`	OpenTelemetry, Tempo, Elasticsearch, Kibana, Filebeat	de făcut
F16	`java/spring/spring-security-keycloak`	OAuth2 / OIDC, realm în Git, resource server	de făcut
F17	`devops/kong-gateway`	Kong DB-less, jwt, rate-limiting, rețele Docker	de făcut
U1 · U2 · U4	`angular-api-calls → rutare + interceptor` · `angular-grafice` · `angular-oidc-keycloak`	rutare, `HttpClient` , interceptor · grafic · OIDC + PKCE	parțial · de făcut · de făcut
U1	`javascript/playwright` (adaptat pe UI-ul lui)	test end-to-end de fum: login → comandă → istoric	de făcut

Unelte de verificare pentru F1–F5

Din `solar-sync-broker-api/` , în Git Bash. Le definești o dată pe sesiune; exemplele din „Gata când” le folosesc fără să le repete (în Git Bash nu ai `jq` , de aceea `grep` + `cut`):

```
B=localhost:8081/api/v1 ; J='Content-Type: application/json'
tok() { curl -s -XPOST $B/auth/login -H "$J" -d "{\"email\":\"$1\",\"password\":\"$2\"}" | grep -o '"token":"[^"]*"' | cut -d'"' -f4; }
M="docker compose exec -T mysql-server mysql -uroot -proot solarsyncbroker_db -e"     # $M "select …"
```

Cine ce poate — enum-ul complet, de la F1

Permisiune	USER (proprietar)	OPERATOR	ADMIN	AGENT	Folosită în
`HOUSE_VIEW`	casa lui	toate	toate	toate	F2, F9
`HOUSE_MANAGE`	–	–	✓	–	F2
`BATTERY_VIEW`	a lui	toate	toate	toate	F2, F3, F9
`BATTERY_CONFIG`	a lui	–	✓	–	F3
`BATTERY_COMMAND`	a lui	toate	toate	–	F3
`TELEMETRY_VIEW`	a lui	toate	toate	toate	F4, F9
`FINANCIAL_VIEW`	a lui	toate	toate	–	F5, F9
`COMMUNITY_VIEW`	–	✓	✓	✓	F4, F5, F9
`GRID_VIEW`	–	✓	✓	✓	F8, F9
`SIMULATION_CONTROL`	–	–	✓	–	F4
`AGENT_VIEW`	–	✓	✓	✓	F10
`AGENT_PROPOSE`	–	–	–	✓	F10
`AGENT_APPROVE`	–	✓	✓	–	F10
`USER_MANAGE`	–	–	✓	–	F1

„a lui” = permisiunea e în token, iar service-ul verifică proprietarul. `AGENT` nu are `BATTERY_COMMAND` : agentul doar **propune**, nu comandă direct (F10). `BATTERY_MANAGE` dispare. De la F16 rolurile vin din Keycloak; harta rol → permisiuni rămâne în API, aceeași.

I. Comunitatea merge

O singură aplicație, `solar-sync-broker-api`. F1–F5 le faci singur: la final ai conturi, case cu baterii, comenzi, o zi simulată și banii casei, toate verificate local. F6 mută testele pe MySQL real, F7–F8 adaugă loguri și metrici; în Grafana vezi comunitatea live.

F1 Contul meu

vizitator · utilizator · administrator

> **Nou: Flyway (V1)** — lecția `spring-flyway` înainte: exercițiile 1–3 le ai făcute (dependențele, baza nouă, prima migrare), termini 4–8 înainte de F1. Restul feature-ului e cu ce știi: Spring Security, JPA, teste cu Mockito și MockMvc.

Ce poate face utilizatorul: vizitatorul își face cont de proprietar și se loghează; oricine e logat își vede contul și permisiunile; administratorul creează conturi `OPERATOR`, `USER` și `AGENT`.

Scenariu.

1. Ana (vizitator) trimite `POST /api/v1/auth/register` cu nume, email, parolă, vârstă și, din greșeală sau cu rea-credință, `"userType":"ADMIN"`. Primește **201** cu `"userType":"USER"` și 6 permisiuni.
2. Ana trimite `POST /auth/login` și primește un token. Cu el, `GET /auth/me` → 200 cu contul ei. Fără token → 401.
3. Adminul (cont creat la pornire din `.env`) se loghează și trimite `POST /api/v1/users` pentru Mihai, `OPERATOR` → 201. Ana încearcă aceeași cerere → 403.

Ce ai deja (rulat pe `f3bd97b`)

- `login`: 200 cu token JWT; email inexistent sau parolă greșită → 401 cu corp complet (`JWTAuthentificationEntryPoint`). `register` salvează parola BCrypt, iar emailul duplicat dă 409.
- **Din Spring Security ai folosit deja** (și planul cere doar atât): `SecurityFilterChain` cu `requestMatchers(...).permitAll()` și `anyRequest().authenticated()`; `@EnableMethodSecurity` + `@PreAuthorize("hasAuthority(...)")` (13 locuri, în 4 controllere); resource server cu `NimbusJwtDecoder` (HS512) și `JwtAuthenticationConverter` (permisiunile din claim-ul `authorities`, fără prefix); `JWTTokenProvider` (jjwt) care pune authorities în token; `AuthenticationManager.authenticate` la login; `AuthenticationEntryPoint` și `AccessDeniedHandler` care scriu JSON; `Principal` în controllere. Nu ai folosit: `hasRole`, `SecurityContextHolder`, `permitAll` pe `/error`, filtru propriu (tokenul îl validează resource server-ul): planul nu cere niciunul.
- `ApplicationIntializer` creează adminul din `ADMIN_EMAIL` / `ADMIN_PASSWORD`, o singură dată. `JWTTokenProvider`, `SecurityConfiguration` (resource server, `SecurityAccessDeniedHandler`), `UserType` cu cele 4 tipuri.
- `AuthControllerIntegrationTests`: 3 teste cu `@SpringBootTest` + MockMvc pe H2 (`application-test.yml`: Flyway oprit, `create-drop`); se loghează cu un cont real, dar niciun test din proiect nu trimite vreodată `Authorization: Bearer` pe o a doua cerere. Cele 5 teste care au nevoie de permisiuni folosesc `@WithMockUser` (`BatteryControllerIntegrationTests`, `FinancialControllerIntegrationTest`).

Ce lipsește sau e stricat (cu `fișier:linie`)

1. **Nu compilează** (CI roșu): `BatteryController.java:43` cheamă `updateConfiguration` cu 2 argumente, metoda cere 3 · `CommandExecutionMapper.java:14` și `:16`: `getHouseId()` și `getTargetSoc()` nu mai există în `CommandExecution` · `BatteryCommandServiceIT.java:99` (aceeași metodă, la test-compile).
2. După ce compilează, `mvn test` = 37 teste, 1 roșu: `BatteryCommandServiceTests.testIncarcareReusita:69` (NullPointerException: `applyCommand` e gol și întoarce `null`; îl implementezi în F3).
3. **B2** · `AuthServiceImpl.java:55-56`: a doua `.userType(...)` suprascrie `USER` cu valoarea din cerere; `AuthReqisterRequest.userType` o primește. Rulat: `register` cu `"userType":"ADMIN"` → 201 cu `HOUSE_MANAGE`.
4. Adminul din `.env` are `"permissions":[]` și token cu `authorities: []` (`ApplicationIntializer.java:41`: builder fără permisiuni): nu poate face nimic.
5. **B6** · permisiunile stau în tabela `user_permissions` (`User.java:57-63`, `@ElementCollection`) și le alege `AuthServiceImpl.java:100` (`permissionsForType`): un `USER` primește doar `HOUSE_VIEW` și `BATTERY_VIEW`. `UserMapper.java:21` le citește de acolo.
6. `UserPermissions` nu are `TELEMETRY_VIEW`, `COMMUNITY_VIEW`, `SIMULATION_CONTROL`, `AGENT_*`; are `BATTERY_MANAGE` și `FINANCIAL_MANAGE`, pe care nu le cere nimeni.
7. **M2** · `UserController.java:28`: `hasAuthority('ADMIN')` (ADMIN e tip de cont, nu permisiune → 403 pentru oricine); calea `/api/v1/user/add`; răspunde 201 fără corp. `UserRequest` n-are parolă și tip (și importă `org.hibernate.usertype.UserType`); `UserCommandServiceImpl.addUser` nu salvează parola, vârsta și tipul.
8. **B5 (corpul)** · `GlobalExceptionHandler.java:30, :39, :52` trimit `apiErrorResponse.error()`, câmp pe care nu l-ai setat → corp gol la 404 / 403 / 409. Rulat: emailul duplicat → 409 fără corp.
9. Nu există handler pentru validare: `register` cu email invalid răspunde **401** (cererea ajunge la `/error`, care cere autentificare), nu 400.
10. `SecurityConfiguration.java:73`: `/api/v1/auth/**` e public, deci `/auth/me` ar fi public; `SecurityConstants.java:18` are în `PUBLIC_URLS` șirul `"http://localhost:8080/login#/"` (warning la pornire: „missing a leading slash"). Parola: `@Size(min = 3)` la `AuthReqisterRequest.java:25`, regula e 8.
11. **Schema**: `application.yml` are `ddl-auto: update` și `flyway.enabled: false`; fișierul `src/main/resources/db/migration/V1_schema.sql` are un **spațiu** în față și un singur `_`, deci Flyway nu-l vede. Conține și tabele fără entitate (`agent`, `agent_seq`), `target_soc` în loc de `power_kw` și `user_permissions`: cu `validate` aplicația n-ar porni.

1. **Compilarea.** `BatteryController`: preia `Principal` și trimite `principal.getName()`. `CommandExecutionMapper`: `houseId` vine din `house`, iar `targetSoc` iese din `CommandExecutionResponse` (în F3 intră `powerKw`). La testul `BatteryCommandServiceIT:99` adaugi al treilea argument. `BatteryCommandServiceTests.testIncarcareReusita`: `@Disabled("F3")`, cu motivul scris; în F3 dispare.
2. **Permisiuni.** `UserPermissions` = cele 14 din matricea de mai sus (ștergi `BATTERY_MANAGE`, `FINANCIAL_MANAGE`). O clasă `RolePermissions` în `system/security` cu `Set<UserPermissions> of(UserType)`, citită de `User.getAuthorities()` și de `UserMapper`. Din `User` dispar câmpul `permissions` și `setPermissions(...)` din `AuthServiceImpl`. `User extends AbstractAuditable` (apare `created_at`; nu se expune în răspuns).
3. **Register (B2).** `AuthReqisterRequest` pierde `userType`; parola `@Size(min = 8, max = 50)`. `AuthServiceImpl.register` pune mereu `UserType.USER` (șterge rândul 56).
4. **Adminul din .env.** `ApplicationIntializer` rămâne; permisiunile vin acum din hartă, nu mai ai ce completa. Un cont `ADMIN` nu se mai creează în niciun alt loc.
5. **Conturi create de admin.** `UserRequest` primește `password` (min 8), `age` (≥ 18), `userType` (`@NotNull`; tipul tău din `users.model`, nu cel din Hibernate). `UserCommandServiceImpl.addUser` salvează parola prin `PasswordEncoder`, vârsta și tipul; `ADMIN` → 400. `UserController`: `POST /api/v1/users` cu `@PreAuthorize("hasAuthority('USER_MANAGE')")`, 201 cu `UserResponse` în corp.
6. **Contul meu.** `GET /api/v1/auth/me` în `AuthController`, cu `Principal` (ca în `HouseController`). În `SecurityConfiguration` ștergi regula `.requestMatchers("/api/v1/auth/**").permitAll()` (rândul 73): `login` și `register` rămân publice prin `PUBLIC_URLS` (rândul de după), iar `/auth/me` cade pe `anyRequest().authenticated()`. Din `PUBLIC_URLS` scoți și șirul greșit. Nu schimbi nimic altceva în Security.
7. **Erorile.** `GlobalExceptionHandler` întoarce `ApiErrorResponse.of(...)` întreg (cu `path` din `HttpServletRequest`), nu `.error()`. Handler nou pentru `MethodArgumentNotValidException` și `HttpMessageNotReadableException` → 400, cu `message` = „câmp: mesaj" pentru prima eroare. Handler-ul de 403 pentru casă vine în F2.
8. **Schema cu Flyway** nou (exercițiile 1–3 din lecție, pe tabelele tale).
   a. Pe `pom.xml`: ai deja `flyway-core` și `flyway-mysql`, iar pe Spring Boot 3.5 (proiectul tău) ajung. `spring-boot-starter-flyway` din lecție există doar de la Boot 4: aici nu-l adaugi.
   b. `application.yml`: ștergi `flyway.enabled: false`, pui `ddl-auto: validate`.
   c. Ștergi `V1_schema.sql` și creezi `V1__schema.sql` (două `_`, fără spațiu), cu cele 6 tabele **exact cum le cer entitățile la finalul F1**: tabelul de mai jos. Cea mai sigură metodă: pe o bază goală, o dată, `ddl-auto: create` + `show create table …`, apoi scrii migrarea și treci pe `validate`. Constrângerile primesc nume (`fk_…`, `uq_…`): V2 le șterge după nume.
   d. `houses.battery_id` și `bateries.house_id` își arată una alteia: creezi întâi `houses` (cu `battery_id`, fără cheia străină), apoi `bateries` (cu cheia străină spre `houses`), apoi `alter table houses add constraint fk_houses_battery …`. Invers, MySQL dă `ERROR 1824: Failed to open the referenced table 'houses'`. F2 desface cercul.
   e. `docker compose down -v` în `solar-sync-broker-api/` (volumul `mysql_data` pleacă; datele sunt generate), apoi `up -d mysql-server`. Flyway nu lucrează pe o schemă creată de Hibernate („non-empty schema").
   f. La pornire: `Migrating schema … to version "1 - schema"`, apoi `Successfully applied 1 migration` și, la a doua pornire, `Schema … is up to date. No migration necessary.` Un `WARN` „Flyway upgrade recommended: MySQL 8.4 is newer…" e normal.
9. **Teste.** Rămân cum le știi: unitare Mockito și `@SpringBootTest` + MockMvc pe H2, cu profilul `test` (`application-test.yml` are `flyway.enabled: false` și `create-drop`, deci tabelele din teste le face Hibernate). V1 conține SQL de MySQL (`datetime(6)`, `bit`), deci testele de aici **nu** îl verifică; îl verifici pornind aplicația pe MySQL, ca la lecție. Singura îmbinare nouă: **login real într-un test**. În `src/test` faci o clasă de ajutor cu două metode: una salvează un cont cu un tip dat (parola codată cu `PasswordEncoder`, ca în `AuthControllerIntegrationTests.setUpUser`), alta trimite `POST /api/v1/auth/login` prin `MockMvc`, citește câmpul `token` din corpul răspunsului (`ObjectMapper`) și îl întoarce. Cererile următoare pun `.header("Authorization", "Bearer " + token)`. E MockMvc și Spring Security, pe care le ai; nou e doar că tokenul se refolosește. **Niciun test nou cu `@WithMockUser`**; cele existente se rescriu în feature-ul lor (F3, F5).
10. **Livrare.** `JWT_SECRET_KEY`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` stau în `.env` pe EC2 și le adaugi în `environment:` din `solar-sync-broker-ops/docker-compose-platform.yaml` (acum nu sunt acolo). Adaugi `.env` în `solar-sync-broker-api/.gitignore` (acum lipsește). Pe EC2: `docker compose down -v` o dată (V1 nu rulează peste o schemă făcută de Hibernate).

Tabelele lui V1 (cum le face Hibernate din entitățile de la finalul F1): `users` (id, first_name, last_name, age, email UQ, password, user_type `VARCHAR(20)`, created_at) · `houses` (id, name, pv_peak_power_kw, max_import_power_kw, max_export_power_kw, enabled, owner_id FK, battery_id FK UQ) · `bateries` (id, soc_percent, max_charge_power_kw, max_discharge_power_kw, efficienty_percent, house_id FK UQ) · `command_executions` (id, house_id FK, command_type, power_kw, status, failure_reason, soc_after_percent, executed_by, created_at) · `telemetry` (id, house_id FK, local_date_time, pv_power_kw, load_power_kw, grid_power_kw, battery_soc_percent, battery_power_kw) · `financial_transaction` (id, house_id, timestamp, transaction_type, amount_kwh, unit_price, total_value). Numele cu greșeli

rămân în V1; fiecare feature le corectează în migrarea lui. **Fără** `user_permissions`, `agent`, `agent_seq`. Hibernate generează `enum(…)` pentru `user_type`; `VARCHAR(20)` trece de `validate` (verificat pe MySQL 8.4).

Endpoint	Permisiune	Cerere	Răspuns
`POST /api/v1/auth/register`	public	`{"firstName":"Ana","lastName":"Pop","email":"ana@solar.ro", "password":"parola123","age":25}`	**201** `{"id":2,"firstName":"Ana","lastName":"Pop","email":"ana@solar.ro", "userType":"USER","permissions": ["HOUSE_VIEW","BATTERY_VIEW", "BATTERY_CONFIG","BATTERY_COMMAND","TELEMETRY_VIEW", "FINANCIAL_VIEW"],"now":"2026-11-12T11:20:05","token":null}`
`POST /api/v1/auth/login`	public	`{"email":"ana@solar.ro","password":"parola123"}`	**200**, același corp ca la register, cu `"token":"eyJhbGciOiJIUzUxMiJ9…"`
`GET /api/v1/auth/me`	orice cont logat	header `Authorization: Bearer …`	**200**, `UserResponse` cu `"token":null`
`POST /api/v1/users`	`USER_MANAGE`	`{"firstName":"Mihai","lastName":"Rus","email":"mihai@solar.ro", "password":"parola123","age":34,"userType":"OPERATOR"}`	**201**, `UserResponse` cu `"id":3`, `"userType":"OPERATOR"`, 9 permisiuni

Erori (toate cu aceeași formă; exemple): **400** `{"timestamp":"2026-11-12T11:20:05","status":400,"error":"BAD_REQUEST","message":"email: Formatul email-ului este invalid","path":"/api/v1/auth/register"}` · **401** la login greșit, `"message":"Trebuie sa fii autentificat pentru a accesa aceasta resursa."` · **403** `"message":"Nu ai permisiunea necesara pentru aceasta actiune.","path":"/api/v1/users"` · **409** `"error":"CONFLICT","message":"User already exists"`.

Reguli

1. Email valid și unic (409). Parolă de minim 8 caractere, vârstă ≥ 18 (400).
2. `register` creează doar `USER`; `userType` din cerere nu există și e ignorat.
3. `POST /users` creează `OPERATOR`, `USER` sau `AGENT`; `ADMIN` → 400 (o excepție nouă + `@ExceptionHandler` în `GlobalExceptionHandler`, mesajul `"Cannot create ADMIN accounts"`). Contul `ADMIN` vine doar din `.env`.
4. Login greșit → 401, același mesaj pentru email inexistent și pentru parolă greșită. Parola nu apare în niciun răspuns.
5. Permisiunile unui cont rezultă din tipul lui, dintr-o singură hartă (matricea de mai sus). Fiecare șir din `hasAuthority(...)` din tot proiectul există în `UserPermissions`: `grep -rn "hasAuthority" src/main` și verifici pe rând (în F2 repari `'house:view'`).
6. Doar `register` și `login` sunt publice; `/auth/me` fără token → 401.

Teste de scris	Ce verifică
`AuthServiceTests.register_alwaysCreatesUserType` (Mockito)	entitatea trimisă la `save` (captor) are `USER` și parola codată
`AuthServiceTests.register_duplicateEmail_throws`	`UserAlreadyexistsException`, fără `save`
`UserCommandServiceTests.addUser_savesPasswordAgeAndType` · `addUser_adminType_rejected`	câmpurile ajung în entitate; `ADMIN` aruncă excepția de 400
`RolePermissionsTests`	câte o asertiune pe tip: `USER` are exact 6; `AGENT` nu are `BATTERY_COMMAND`; doar `ADMIN` are `USER_MANAGE`
`AuthControllerIntegrationTests` (MockMvc + login real)	`register_withAdminInBody_createsUser` · `register_invalidEmail_returns400WithBody` · `register_duplicate_returns409WithBody` · `me_withoutToken_returns401` · `me_withToken_returnsAccount`
`UserControllerIntegrationTests`	`addUser_asAdmin_returns201WithBody` · `addUser_asUser_returns403` · `addUser_asOperator_returns403`; tokenul adminului vine din `/login`, nu dintr-un utilizator inventat
`AdminBootstrapTests` (`@SpringBootTest(properties=…)` cu `ADMIN_EMAIL`)	adminul din `.env` se loghează și tokenul lui conține `USER_MANAGE`

Gata când (local, în `solar-sync-broker-api/`; aceleași variabile le refolosești în F2–F5):

```
B=localhost:8081/api/v1 ; J='Content-Type: application/json'
tok() { curl -s -XPOST $B/auth/login -H "$J" \
  -d "{\"email\":\"$1\",\"password\":\"$2\"}" | grep -o '"token":"[^"]*"' | cut -d'"' -f4; }

docker compose down -v && docker compose up -d mysql-server
ADMIN_EMAIL=admin@solar.ro ADMIN_PASSWORD='Admin1234!' mvn spring-boot:run
  → Successfully applied 1 migration to schema `solarsyncbroker_db`, now at version v1
docker compose exec -T mysql-server mysql -uroot -proot solarsyncbroker_db \
  -e "select version,description,success from flyway_schema_history; show tables"
  → 1  schema  1   și 7 tabele (cele 6 + flyway_schema_history)

curl -s -XPOST $B/auth/register -H "$J" -d '{"userType":"ADMIN","firstName":"Ana",
 "lastName":"Pop","email":"ana@solar.ro","password":"parola123","age":25}'
  → 201, "userType":"USER", 6 permisiuni
ANA=$(tok ana@solar.ro parola123) ; ADMIN=$(tok admin@solar.ro 'Admin1234!')
curl -s -o /dev/null -w '%{http_code}\n' $B/auth/me                          → 401
curl -s -o /dev/null -w '%{http_code}\n' $B/auth/me -H "Authorization: Bearer $ANA"   → 200
curl -s -o /dev/null -w '%{http_code}\n' -XPOST $B/users -H "$J" -H "Authorization: Bearer $ANA" \
  -d '{"firstName":"M","lastName":"R","email":"m@solar.ro","password":"parola123","age":34,"userType":"OPERATOR"}'
  → 403   (la fel cu $ADMIN → 201; cu "userType":"ADMIN" → 400)
curl -s -XPOST $B/auth/register -H "$J" -d '{"firstName":"A","lastName":"B","email":"nu-e-email",
 "password":"parola123","age":25}'   → 400, "message":"email: Formatul email-ului este invalid"
mvn test   → BUILD SUCCESS, Failures: 0, Errors: 0, Skipped: 1 (cel cu @Disabled("F3"))
```

Pe EC2: login cu adminul din `.env` → 200, apoi `/auth/me` = 200.

Capcane

- **Numele fișierului.** `V1_schema.sql` (un `_`) sau cu spațiu în față: Flyway scrie doar un `WARN` „No migrations found”, iar `validate` pică cu „missing table […]”. Verifici cu `select * from flyway_schema_history`.
- **Baza veche.** Pornești Flyway pe o bază făcută de Hibernate: se oprește cu „Found non-empty schema(s) … but no schema history table” (exercițiul 1). Local și pe EC2: `down -v`, o singură dată.
- **400 care arată ca 401.** Fără handler pentru validare, cererea merge la `/error`, care cere token. Fixul e un `@ExceptionHandler` în `GlobalExceptionHandler` (ca cele care există), nu o regulă nouă în Security. Fără handler, în `curl` vezi 401 cu `"path":"/error"`; în MockMvc nu există dispatch la `/error`, deci testul vede 400 fără corp și pică pe aserțiunea pe `message`.

F2 Casa cu bateria ei

administrator · operator · proprietar

Cu ce știi: da. A doua migrare (`V2`) e exercițiul 5 din lecția `spring-flyway` (`ALTER TABLE`, o migrare rulată nu se mai editează): începi F2 după ce l-ai terminat.

Ce poate face utilizatorul: administratorul adaugă, modifică și șterge case, iar bateria apare odată cu casa; proprietarul își vede casa și bateria; operatorul și administratorul le văd pe toate.

Scenariu.

1. Adminul trimite `POST /api/v1/houses` pentru „Casa Pop”, proprietar Ana (id 2). Primește 201 cu `"id":1`.
2. Ana cere `GET /api/v1/houses/1/battery` și primește bateria implicită: 10 kWh, SoC 50%, 3 kW / 3 kW, randament 95%.
3. Dan (alt cont `USER`) cere `GET /api/v1/houses/1` → 403 cu corp complet. Oricine cere `/houses/999` → 404 cu corp complet.
4. Adminul trimite `PATCH /houses/1 {"enabled":false}`: casa iese din simulare, dar rămâne în sistem.

Ce ai deja

- `HouseController` cu cele 6 operații; `createHouse` verifică numele unic (409) și proprietarul (404, `UserNotFoundException`); `HouseRepository.existsByName` și `existsByIdAndOwner_Email`.

- `HouseQueryServiceImpl.checkHouseAccess` (ADMIN și OPERATOR trec, proprietarul se verifică în SQL), metoda care trebuie să decidă accesul peste tot, dar nu o cheamă decât ea însăși.
- `GET /houses/{id}/battery` merge pentru proprietar și admin (`BatteryQueryServiceImpl`). 21 de teste pe case: `HouseCommandServiceTests` (8), `HouseCommandServiceIT` (7), `HouseQueryServiceIT` (6), pe H2 (cele două clase `…IT` nu rulează la `mvn test`, vezi mai jos).

Ce lipsește sau e stricat

1. **B1** · `HouseController.java:33, :39, :47, :54, :60, :66`: `'house:view'` / `'house:manage'` nu există în token (`HOUSE_VIEW`, `HOUSE_MANAGE`). Rulat: ADMIN → `GET /houses` 403.
2. **B8** · `HouseCommandServiceImpl.java:38–52`: `Battery` se construiește, dar nu e legată de casă (builder-ul casei n-o primește) → casa se salvează fără baterie. `:42`: `efficientyPercent(0.95)`, iar validatorul împarte la 100 (așteaptă 95.0: `BatteryCommandValidatorImpl.java:37`).
3. `House.java:39–41` (`@JoinColumn(name="battery_id")`) + `Battery.java:35–37` (`house_id`): două chei străine, una spre alta. Rămâne `Battery.house`; pe casă devine `mappedBy`.
4. **B5 (accesul)** · `HouseQueryServiceImpl.java:45` lasă doar ADMIN, deci OPERATOR primește 403; `:42` aruncă `IllegalArgumentException` (M4 → 500); `:28–32`: `getAllHouses` dă toate casele oricui. Excepția aruncată în servicii e `HouseAccessDeniedHandler` (e excepție, nu handler), iar `GlobalExceptionHandler.java:33` prinde `AccessDeniedExceptions`, pe care n-o aruncă nimeni → 500.
5. `BatteryQueryServiceImpl.java:32` (doar ADMIN) și `BatteryCommandServiceImpl.java:50–59` (`validateHouseAccess`) repetă verificarea; `HouseController.java:41` are un `System.out.println(principal)` (M5).
6. `deleteHouse` (`HouseCommandServiceImpl.java:105–110`) șterge direct: pe o casă cu comenzi, FK-ul dă 500. `HouseRequest` n-are `enabled`, deci casa nu se poate scoate din simulare.
7. **B9** (se repară în F6) · cele 16 teste din `HouseCommandServiceIT`, `HouseQueryServiceIT` și `BatteryCommandServiceIT` nu rulează la `mvn test` (numele nu se potrivește cu ce caută surefire). Până în F6 le rulezi pe nume: `mvn test -Dtest=HouseCommandServiceIT,HouseQueryServiceIT,BatteryCommandServiceIT`.
8. Rulate pe nume, pe H2, cele 3 teste din `BatteryCommandServiceIT` dau 2 roșii (`testIncarcare:84`, `testDescarcare:93`): așteaptă un `applyCommand` care încă nu există (F3). Al treilea, `testActualizareConfig:102–103`, trece pentru motivul greșit (aserțiunea e 3.0, valoarea de dinainte). Cele 2 roșii rămân roșii până în F3.

1. **Bază.** `V2__house_battery.sql`, în același commit cu entitățile: `houses` pierde `battery_id` (întâi cheia străină și indexul unic după numele din V1, apoi coloana) și primește `created_at` și un unic pe `name`; `bateries` → `batteries`; `efficienty_percent` → `efficiency_percent`; `capacity_kwh NOT NULL` (10.0 pentru rândurile existente). În cod: `@Table(name="batteries")`, câmpul `efficiencyPercent` (refactor de IDE în tot proiectul, teste incluse), `@Column(unique = true)` pe `House.name`, `capacityKwh` (și în `BatteryResponse`, copiat de `BatteryMapper`, ca `GET /battery` să-l arate), `House extends AbstractAuditable`, `@OneToOne(mappedBy = "house", cascade = ALL, orphanRemoval = true)` pe `House.battery`, `@JoinColumn(name = "house_id")` pe `Battery.house`.
2. **Service.** `createHouse`: bateria implicită (10.0 kWh, SoC 50.0, 3.0 / 3.0 kW, randament **95.0**) cu `battery.setHouse(house)` și `house.setBattery(battery)`, un singur `save`, în aceeași tranzacție. `deleteHouse`: `delete` + `flush`, iar `DataIntegrityViolationException` devine `HouseHasHistoryException` (409): merge singură și pentru comenzile din F3, telemetria din F4, banii din F5. `patchHouse` tratează și `enabled` (`HouseRequest` primește `Boolean enabled`).
3. **Accesul, într-un singur loc.** `checkHouseAccess(email, houseId)` (există) e singura verificare: o cheamă `getHouseForCaller`, `BatteryQueryServiceImpl`, `BatteryCommandServiceImpl` (F3), telemetria (F4), banii (F5). Casă inexistentă → 404 înaintea verificării de proprietar. `getAllHouses(email)`: ADMIN și OPERATOR toate, `USER` doar ale lui (`findByOwner_Email`). Dispar `validateHouseAccess`, `IllegalArgumentException` (→ `UserNotFoundException`) și `println`.
4. **Erori.** O singură excepție de acces (păstrezi `HouseAccessDeniedHandler` sau o redenumești) și un `@ExceptionHandler` pe ea → 403 cu `ApiErrorResponse` întreg; scoți handler-ul mort pentru `AccessDeniedExceptions`. Mesaj nou în `ErrorConstants`: `HOUSE_HAS_HISTORY`.
5. **Intrare.** Cele 6 operații din `openapi.yaml` și `GET /houses/{houseId}/battery`, cu `HOUSE_VIEW`, `HOUSE_MANAGE`, `BATTERY_VIEW` (în `HouseController`: toate șirurile devin `HOUSE_*`).
6. **Test.** Matricea cu trei conturi reale (admin, proprietar, străin), cu tokenul din `/login` (ajutorul de test din F1), în `@SpringBootTest` + MockMvc, pe H2. Aceeași H2 e comună tuturor claselor: fiecare clasă își curăță datele în `@BeforeEach` cu `deleteAll`, în ordinea cheilor străine (`command_executions`, `telemetry`, `financial_transactions`, `bateries`, `houses`, `users`), **chiar dacă e `@Transactional`**: o clasă `@Transactional` nu vede curat rândurile lăsate de clasele care nu sunt. Aceeași ștergere o pui și în `BatteryCommandServiceIT.setup` din F3, altfel testele pică în funcție de ordinea claselor. V2 nu e verificată de testele astea (tabelele le face Hibernate); o verifici pornind aplicația pe MySQL.

Testele existente pe care le rupe F2 (le repari în același commit)

- `BatteryControllerIntegrationTests.setUp` (bateria la `:74`) și `BatteryCommandServiceIT.setup` (bateria la `:68`) construiesc bateria de mână: au nevoie de `.capacityKwh(10.0)` (câmpul e `@NotNull`).
- `BatteryControllerIntegrationTests.prorietardacommandainvalida400` (`:103`) creează o a doua casă cu același nume: dă duplicat pe `uq_houses_name`.
- `Boolean enabled` în `HouseRequest` rupe cele 14 `new HouseRequest(...)` din `HouseControllerTest`, `HouseCommandServiceTests` și `HouseCommandServiceIT`.
- `getAllHouses(email)` rupe `HouseQueryServiceIT:55` și `HouseQueryServiceTests:61`.
- `checkHouseAccess` în locul verificărilor de azi rupe `BatteryQueryServiceTests.testGetBatteryByHouseId` și `HouseQueryServiceTests.testGetHouseForCaller` (mock-urile se schimbă).

Endpoint	Permisiune	Cerere → răspuns

`POST /api/v1/houses`	`HOUSE_MANAGE`	`{"name":"Casa Pop","pvPeakPowerKw":6.5,"maxImportPowerKw":10.0, "maxExportPowerKw":8.0,"ownerId":2}`
		201
		`{"id":1,"name":"Casa Pop","pvPeakPowerKw":6.5,"maxImportPowerKw":10.0, "maxExportPowerKw":8.0,"enabled":true,"ownerId":2}`

`GET /api/v1/houses`	`HOUSE_VIEW`	**200**: lista; Ana primește doar `[{"id":1,…}]`, Dan primește `[]`, adminul și operatorul toate casele
`GET /houses/{id}`	`HOUSE_VIEW` + proprietar	**200**, aceeași formă ca la POST
`PUT` · `PATCH /houses/{id}`	`HOUSE_MANAGE`	PUT: corpul de la POST, **200** · PATCH `{"enabled":false}` → **200** cu `"enabled":false`
`DELETE /houses/{id}`	`HOUSE_MANAGE`	**204**, fără corp
`GET /houses/{houseId}/battery`	`BATTERY_VIEW` + proprietar	**200** `{"id":1,"capacityKwh":10.0,"socPercent":50.0,"maxChargePowerKw":3.0, "maxDischargePowerKw":3.0,"efficiencyPercent":95.0,"houseId":1}`

Erori: **400** `"message":"pvPeakPowerKw: pvPeakPowerKw trebuie sa fie mai mare ca 0"` · **403** `{"timestamp":"2026-11-12T11:25:40","status":403,"error":"FORBIDDEN","message":"House access denied","path":"/api/v1/houses/1"}` · **404** `"status":404,"error":"NOT_FOUND","message":"House not found","path":"/api/v1/houses/999"` (la fel „User not found" pentru `ownerId` inexistent) · **409** `"message":"House already exists"` și, la ștergere, `"message":"House has history"`.

Reguli

1. Nume obligatoriu și unic (400 / 409, și ca `UNIQUE` în bază); puterile > 0 (400); proprietar inexistent → 404.
2. O casă = o baterie, creată în aceeași tranzacție: dacă pică una, nu rămâne nimic. Bateria implicită: 10 kWh, SoC 50%, 3 kW încărcare / 3 kW descărcare, randament 95%.
3. Acces: ADMIN și OPERATOR la orice casă; `USER` doar la casa lui; altfel 403. Casă inexistentă → 404 (nu 403).
4. Casa cu istoric (comenzi, telemetrie, tranzacții) nu se șterge (409); o scoți din simulare cu `enabled: false`.
5. `GET /houses`: `USER` vede doar casele lui (poate fi lista goală); ADMIN și OPERATOR le văd pe toate.

Teste de scris	Ce verifică
`HouseCommandServiceTests.createHouse_attachesDefaultBattery` (Mockito)	captor pe `save`: bateria are 10.0 / 50.0 / 3.0 / 3.0 / 95.0 și `battery.getHouse()` e casa
`HouseCommandServiceTests.deleteHouse_withHistory_throwsConflict`	`DataIntegrityViolationException` la `flush` → `HouseHasHistoryException`
`HouseQueryServiceTests.getAllHouses_ownerSeesOnlyOwn` · `_operatorSeesAll`	metoda de repository aleasă după tipul contului
`HouseControllerIntegrationTests.accessMatrix` (H2, login real)	pe fiecare din cele 6 operații, cu admin / proprietar / străin: admin 200 · 200 · 201 · 200 · 200 · 204; proprietar 200 · 200 · 403 · 403 · 403 · 403; străin 200 (`[]`) · 403 · 403 · 403 · 403 · 403
`HouseControllerIntegrationTests.createHouse_thenGetBattery_returnsDefaults`	după POST, `GET /battery` → 200 cu `efficiencyPercent 95.0`
`HouseControllerIntegrationTests.errorBodies_areComplete`	403, 404 și 409 au `status`, `error`, `message`, `path`

Gata când (bază nouă, ca în F1; Dan = `dan@solar.ro`, cont `USER` făcut prin `register`):

```
mvn test          → BUILD SUCCESS, Failures: 0, Errors: 0, Skipped: 1 (cel cu @Disabled("F3"))
mvn test -Dtest=HouseCommandServiceIT,HouseQueryServiceIT   → Failures: 0, Errors: 0
docker compose down -v && docker compose up -d mysql-server
ADMIN_EMAIL=admin@solar.ro ADMIN_PASSWORD='Admin1234!' mvn spring-boot:run
  → Migrating … to version "2 - house battery", Successfully applied 2 migrations … now at version v2
ADMIN=$(tok admin@solar.ro 'Admin1234!')      # plus ANA și DAN, ca în F1
curl -s -XPOST $B/houses -H "$J" -H "Authorization: Bearer $ADMIN" -d '{"name":"Casa Pop",
 "pvPeakPowerKw":6.5,"maxImportPowerKw":10.0,"maxExportPowerKw":8.0,"ownerId":2}'   → 201, "id":1
curl -s $B/houses/1/battery -H "Authorization: Bearer $ANA"      → 200, "socPercent":50.0 ... "efficiencyPercent":95.0
curl -s -w ' %{http_code}\n' $B/houses/1 -H "Authorization: Bearer $DAN"  → corp cu "House access denied" + 403
curl -s -w ' %{http_code}\n' $B/houses/999 -H "Authorization: Bearer $ADMIN" → "House not found" + 404
curl -s $B/houses -H "Authorization: Bearer $DAN"                → []
docker compose exec -T mysql-server mysql -uroot -proot solarsyncbroker_db \
  -e "select count(*) from batteries; select version from flyway_schema_history order by installed_rank desc limit 1"
  → 1   și   2
```

Capcane

- **Entitate schimbată, migrare uitată.** Adaugi `capacityKwh` și nu scrii V2: aplicația nu pornește, „Schema-validation: missing column [capacity_kwh] in table [bateries]" (exercițiul 4).
- **Un singur _.** `V2_house_battery.sql` e ignorat fără niciun avertisment; aceeași eroare ca mai sus (exercițiul 5). Numele corect: `V2__house_battery.sql`.
- **Editezi V1 sau V2 după ce a rulat.** „Migration checksum mismatch for migration version 1". O migrare rulată nu se mai editează: local faci `down -v`, pe EC2 adaugi o migrare nouă.

F3 Comand bateria — poarta

proprietar · operator · administrator

Cu ce știi: da. Spring Data JPA, validări, teste Mockito, `@Transactional`. Nicio unealtă nouă.

Ce poate face utilizatorul: proprietarul își vede bateria, îi schimbă puterile maxime, îi trimite `CHARGE`, `DISCHARGE` sau `IDLE` cu puterea în kW și vede istoricul comenzilor, fiecare cu rezultatul ei.

Scenariu (casa 1 a Anei: 10 kWh, SoC 50%, max 3 / 3 kW, randament 95%)

1. Ana trimite `CHARGE` 2 kW → 201 `SUCCESS`, SoC 69.0 (2 kW × 1 h × 0.95 = 1.9 kWh = +19 puncte).
2. Ana trimite `CHARGE` 90 kW → 201 `REJECTED`, `BATTERY_POWER_LIMIT`, SoC tot 69.0: comanda respinsă se salvează, bateria nu se mișcă.
3. Încă un `CHARGE` 2 kW → `SUCCESS`, SoC 88.0. Încă unul → 88 + 19 = 107 > 90 → `REJECTED`, `BATTERY_SOC_LIMIT`.
4. Ana trimite `DISCHARGE` 2 kW → `SUCCESS`, SoC 66.9474 (2 / 0.95 = 2.1053 kWh = −21.05 puncte). Adminul schimbă puterile la 4.5 / 4.0 kW; Dan (străin) încearcă orice → 403.

Ce ai deja

- `BatteryCommandValidatorImpl` (4 teste verzi): putere peste maxim → `BatteryPowerLimitException`; SoC ieșit din 10–90% → `BatterySocLimitException`. `EnergyEngineImpl` (3 teste).
- `BatteryController` cu `getBattery`, `updateConfiguration`, `sendCommand`, `getCommands` (compilează după F1). `CommandExecution` are deja `power_kw`, `failure_reason`, `soc_after_percent`, `executed_by`. Excepțiile bateriei, `BatteryAction`, `BatteryCommand(action, powerKw)`.

Ce lipsește sau e stricat

1. `BatteryCommandServiceImpl.java:62–80`: `updateConfiguration`, `applyCommand`, `executeCommand` întorc `null`, `getCommandsForHouse` întoarce `List.of()` (corpurile s-au șters în `db0db04`). Câmpurile `repository`, `validator`, `houseRepository` există, nefolosite.
2. **B4** · `BatteryCommandRequest.java:11–14`: `targetSoc` (procent 0–100) în loc de putere în kW; **M3** · `:16–17`: `houseId` și în corp, și în cale. `CommandExecutionResponse.java:9`: `targetSoc`, fără `failureReason` și `socAfterPercent`. `BatteryConfigurationRequest.java:7–10`: doar `@NotNull`, nu `@Positive`.
3. `BatteryCommandValidatorImpl.java:15`: `DEFAULT_CAPACITY_KWH = 10.0` constantă; capacitatea trebuie să vină din baterie (`capacityKwh`, adăugat în F2). Limitele 10 / 90 sunt scrise de două ori (`:16–17` și `BatteryCommandServiceImpl.java:28–29`).
4. **B6** · proprietarul are `BATTERY_CONFIG` și `BATTERY_COMMAND` din harta F1; verificarea „e casa lui?" trebuie să ruleze (acum n-are cum).
5. **B3** · `BatteryCommandServiceIT.testActualizareConfig:102–103` afirmă 3.0, adică „PATCH nu salvează". `testIncarcare` / `testDescarcare` (roșii când rulezi clasa pe nume) și `BatteryCommandServiceTests.testIncarcareReusita` (`@Disabled("F3")`) așteaptă `applyCommand`.

1. **Bază.** `V3__commands.sql`: `command_type` și `status` `VARCHAR(20)`, `failure_reason` `VARCHAR(50)`, `soc_after_percent` `NOT NULL` și un index pe `(house_id, created_at)` pentru istoric. Aceleași lungimi și `nullable = false` pe `CommandExecution`, în același commit.
2. **Fizica, într-un singur loc.** Limitele 10 și 90 într-o clasă de constante citită de validator și de `applyCommand`; capacitatea din `battery.getCapacityKwh()`. Testele validatorului trec baterii cu capacitate.
3. **applyCommand(battery, command, deltaHours).** Cheamă validatorul, apoi: `CHARGE` → SoC + putere × ore × (randament / 100) / capacitate × 100; `DISCHARGE` → SoC − putere × ore / (randament / 100) / capacitate × 100; `IDLE` nu schimbă nimic. Salvează bateria și o întoarce.
4. **executeCommand(houseId, request, username)** (o singură cale; F10 și F11 o refolosesc, nu se scrie alta). Verifică accesul cu `checkHouseAccess` (F2), ia bateria casei, cheamă `applyCommand` cu `deltaHours = 1.0`. Reușită → rând `SUCCESS` cu `socAfterPercent`. Prinde **doar** `BatteryPowerLimitException` și `BatterySocLimitException` → rând `REJECTED` cu `failureReason` = `BATTERY_POWER_LIMIT` / `BATTERY_SOC_LIMIT` și SoC-ul neschimbat. `executedBy` = emailul din token. Metoda e `@Transactional` (fără `readOnly`).
5. **updateConfiguration(houseId, request, username).** Verifică accesul, setează ambele puteri, salvează. `@Transactional` normal. `getCommandsForHouse`: acces, apoi cele mai noi primele (`findByHouseIdOrderByCreatedAtDesc`).
6. **DTO-uri.** `BatteryCommandRequest(commandType, powerKw)` cu `@NotNull` și `@PositiveOrZero` pe putere; `CommandExecutionResponse` primește `powerKw`, `failureReason`, `socAfterPercent` în locul lui `targetSoc`; `BatteryConfigurationRequest` cu `@Positive` (`capacityKwh` în `BatteryResponse` există din F2). `CommandExecutionMapper` se aliniază.
7. **Intrare.** Cele 4 endpoint-uri din tabel; `houseId` doar din cale. `CHARGE` / `DISCHARGE` cu putere 0 → 400. E o regulă între două câmpuri, deci nu o scrii cu o adnotare pe un câmp: o metodă `isPowerValid()` cu `@AssertTrue` în record.
8. **Test.** Fiecare test de comandă verifică **SoC-ul de după și statusul**, nu doar codul HTTP. Scoți `@Disabled("F3")` de la `testIncarcareReusita`, corectezi aserțiunea de la `testActualizareConfig` (4.5 și 4.0) și rulezi pe nume `BatteryCommandServiceIT`: toate 3 verzi.

Testele existente pe care le rupe F3

- `BatteryCommandServiceTests`: bateria din `setup` are nevoie de `.capacityKwh(10.0)`, altfel formula împarte la `null`.
- Cele 2 teste `@WithMockUser` din `BatteryControllerIntegrationTests` se rescriu pe login real, cu cererea nouă (`powerKw` în loc de `targetSoc`, fără `houseId` în corp).

Endpoint	Permisiune	Cerere → răspuns

`POST /api/v1/houses/{houseId}/battery/commands`	`BATTERY_COMMAND` + proprietar	`{"commandType":"CHARGE","powerKw":2.0}`
		201
		`{"id":1,"houseId":1,"commandType":"CHARGE","powerKw":2.0, "status":"SUCCESS","failureReason":null,"socAfterPercent":69.0, "executedBy":"ana@solar.ro","createdAt":"2026-11-12T11:30:12"}`
		Respinsă (`"powerKw":90.0`): tot **201**, `"status":"REJECTED","failureReason":"BATTERY_POWER_LIMIT","socAfterPercent":69.0`

`GET …/battery/commands`	`BATTERY_COMMAND` + proprietar	**200**, listă de obiecte ca mai sus, cele mai noi primele
`PATCH …/battery/configuration`	`BATTERY_CONFIG` + proprietar	`{"maxChargePowerKw":4.5,"maxDischargePowerKw":4.0}` **200**, `BatteryResponse` cu `4.5` și `4.0`; `GET …/battery` arată aceleași valori
`GET …/battery` (din F2)	`BATTERY_VIEW` + proprietar	**200**, cu `capacityKwh` (adăugat în F2)

Erori: **400** `"message":"commandType: Tipul comenzii nu poate fi nul"` (și 400 la `"commandType":"CHARGEX"`, prin handler-ul de JSON ilizibil din F1) · **403** `"message":"House access denied","path":"/api/v1/houses/1/battery/commands"` · **404** `"message":"House not found"`.

Reguli

1. Proprietarul doar la casa lui; `ADMIN` și `OPERATOR` la orice casă; străinul → 403. Casă inexistentă → 404.
2. O comandă acționează un tick (o oră virtuală, `deltaHours = 1.0`).
3. Putere peste maximul bateriei → `REJECTED` / `BATTERY_POWER_LIMIT`. SoC-ul de după în afara 10–90% → `REJECTED` / `BATTERY_SOC_LIMIT`. Bateria nu se schimbă, răspunsul rămâne 201: respingerea e un rezultat salvat, nu o eroare HTTP.
4. Orice altă eroare nu e `REJECTED`: iese ca eroare (404, 403, 500), fără rând salvat.
5. `CHARGE` / `DISCHARGE` cu `powerKw` 0 sau negativ → 400. `IDLE` ignoră puterea.
6. Puterile maxime din configurare sunt > 0 (400).
7. Istoricul: cele mai noi primele; o casă cu comenzi nu se șterge (409, din F2).

Teste de scris	Ce verifică
`BatteryCommandServiceTests.applyCommand_charge2kw_soc69` (Mockito)	baterie 10 kWh, SoC 50, randament 95: după `CHARGE` 2 kW → 69.0 (delta 0.0001); validatorul e chemat
`…applyCommand_discharge2kw_from69_soc47_9474`	69.0 → 47.9474 (delta 0.001)
`…executeCommand_powerLimit_savesRejected_soc69` · `…socLimit_savesRejected`	rândul salvat are `REJECTED`, motivul și SoC-ul neschimbat
`…executeCommand_unexpectedException_isNotRejected`	un `RuntimeException` oarecare iese din metodă; `save` pe comenzi nu e chemat
`…updateConfiguration_savesBothValues` · `…stranger_throwsAccessDenied`	valorile noi ajung la `save`; străinul nu
`BatteryControllerIntegrationTests` (H2, login real)	`ownerCharges2kw_returns201Success_soc69` · `ownerCharges90kw_returns201Rejected_socStill69` (apoi `GET /battery` → 69.0) · `socLimit_rejected` (al treilea `CHARGE` 2 kW) · `patchConfig_thenGetShowsNewValues` · `stranger_gets403_onAllFour` · `operator_canCommandAnyHouse` · `history_newestFirst` · `deleteHouseWithCommands_returns409` (clasa **fără** `@Transactional`: într-o clasă `@Transactional` ștergerea dă 500 cu `TransientObjectException`, nu cheia străină)

Gata când (casa 1 din F2, SoC 50; $ANA, $DAN, $ADMIN ca în F1):

```
C=$B/houses/1/battery/commands
curl -s -XPOST $C -H "$J" -H "Authorization: Bearer $ANA" -d '{"commandType":"CHARGE","powerKw":2.0}'
  → 201 SUCCESS  socAfterPercent 69.0
… "powerKw":90.0 → 201 REJECTED BATTERY_POWER_LIMIT  socAfterPercent 69.0
… "powerKw":2.0  → 201 SUCCESS 88.0       … "powerKw":2.0 → 201 REJECTED BATTERY_SOC_LIMIT 88.0
… {"commandType":"DISCHARGE","powerKw":2.0} → 201 SUCCESS ≈ 66.9474
curl -s -XPATCH $B/houses/1/battery/configuration -H "$J" -H "Authorization: Bearer $ANA" \
  -d '{"maxChargePowerKw":4.5,"maxDischargePowerKw":4.0}'  → 200 ; GET .../battery → 4.5 și 4.0
curl -s $C -H "Authorization: Bearer $ANA"   → 5 obiecte, ids 5,4,3,2,1
curl -s -o /dev/null -w '%{http_code}\n' -XPOST $C -H "$J" -H "Authorization: Bearer $DAN" \
  -d '{"commandType":"IDLE","powerKw":0}'    → 403
curl -s -w ' %{http_code}\n' -XDELETE $B/houses/1 -H "Authorization: Bearer $ADMIN"
  → {"…","status":409,"message":"House has history",…} 409
docker compose exec -T mysql-server mysql -uroot -proot solarsyncbroker_db \
  -e "select status,count(*) from command_executions group by status"  → REJECTED 2 / SUCCESS 3
mvn test -Dtest=BatteryCommandServiceIT,HouseCommandServiceIT,HouseQueryServiceIT   → Failures: 0, Errors: 0
mvn test ; grep -rn "@Disabled" src/test      → BUILD SUCCESS, Skipped: 0 ; grep nu găsește nimic
```

Capcane

- **`catch (Exception e)` pe tot.** Orice eroare ajunge `REJECTED` fără motiv, iar un 500 real arată ca o respingere normală. Prinzi cele două excepții ale validatorului, nimic mai mult.
- **Excepția iese din metoda `@Transactional`.** Tranzacția se marchează „rollback-only” și rândul `REJECTED` nu se mai salvează. Prinzi excepția **înăuntru**, înainte de `save`.
- **`readOnly = true` pe o metodă care scrie.** Răspunsul arată valorile noi (obiectul din memorie), baza nu primește niciun UPDATE. Testul care verifică doar răspunsul trece; cel care citește din bază pică.

F4 O zi simulată

administrator · proprietar · operator

Cu ce știi: da. Tick-ul pornește la cerere, printr-un endpoint, nu automat; logul e `log.info` simplu. Nicio unealtă nouă.

Ce poate face utilizatorul: cu profilul `demo`, la prima pornire există 10 case cu proprietarii lor; administratorul rulează un tick (o oră virtuală) și fiecare casă activă primește producție PV, consum, preț și un rând de telemetrie; proprietarul își vede ultimele măsurători, operatorul vede starea comunității.

Scenariu

1. Pornești aplicația cu `SPRING_PROFILES_ACTIVE=demo` pe o bază goală: apar `owner1@solar.ro` … `owner10@solar.ro` (parola `Demo1234!`) și „Casa 1” … „Casa 10”, fiecare cu bateria implicită. Casa *i* are `pvPeakPowerKw` = 4.0 + (*i* mod 3): 5, 6, 4, 5, 6, 4, 5, 6, 4, 5 kW, în total 50 kW.
2. Adminul trimite `POST /api/v1/simulation/tick` de 13 ori (de fiecare dată 201). Primul tick e la 2026-06-01 00:00 (noapte: PV 0), al 13-lea la 12:00 (prânz: PV = vârful casei).
3. `owner1` cere `GET /api/v1/houses/1/telemetry` și primește o listă cu cele 13 rânduri, cele mai noi primele (cel mult 50). `owner2` pe aceeași casă → 403.
4. Operatorul cere `GET /api/v1/community/state`: 10 case, PV total 50.0 kW, SoC mediu 50.0, tick 13, `PAUSED` (nu rulează nimic singur).

Ce ai deja

- `EnergyEngineImpl.compute(pv, load, batteryPowerKw, deltaHours)` + 3 teste: întoarce import / export în kW și kWh. `Telemetry` (entitate, fără `price`), `SimulationProperties` (seed 42, `tickSeconds` 60, `realToVirtualRatio` 60; înregistrat prin `@ConfigurationPropertiesScan`, nefolosit), `ClockConfig` (un `Clock` UTC, nefolosit).
- Folderele goale `simulation/{controller,service,…}`; DTO-urile `ComunityStateResponse`, `HouseTelemetryPageResponse`, `TelemetryResponse`.

Ce lipsește sau e stricat

1. `PvGenerator.java` și `TelemetryRepository.java` sunt tot fișierul comentat; `LoadGenerator.java:3` și `SimulationClock.java:3` sunt clase goale; nu există generator de preț, `SimulationService`, controller de simulare sau de telemetrie, seeder.
2. `Telemetry.java:33–34`: câmpul `localDateTime` (coloana `local_date_time`), fără preț; metoda comentată din repository vorbește deja de `RecordedAt`.
3. `TelemetryResponse.java:6`: `content` e `List<TelemetryResponse>`, adică se conține pe sine, iar contractul (`openapi.yaml`) cere oricum o listă simplă: clasa se șterge. Rândul de telemetrie e `HouseTelemetryPageResponse` (numele spune „page”, dar e un rând); ea are `localDateTime` și n-are prețul.
4. Nimic nu e pornit automat și nimic nu cere asta: nu există nicio programare periodică în cod. Un tick e o cerere HTTP.

1. **Bază.** `V4__telemetry.sql`: `local_date_time` → `recorded_at`, `price_per_kwh DECIMAL(10,4) NOT NULL` (pe un tabel cu rânduri, adaugi coloana cu `DEFAULT` și scoți `DEFAULT` după), unic pe `(house_id, recorded_at)`. În cod: `Telemetry.recordedAt` și `BigDecimal pricePerKwh`.
2. **Fusul orar.** URL-ul JDBC din `application.yml` are `serverTimezone=UTC`. Cu JVM-ul pe EET, un `LocalDateTime` se scrie în bază cu –3 h, iar interogările cu `recorded_at='2026-06-01 12:00:00'` din „Gata când” dau gol. În URL (în `application.yml`, în `docker-compose.yaml` din `solar-sync-broker-api/` și în `docker-compose-platform.yaml` din `-ops`) înlocuiești `serverTimezone=UTC` cu `connectionTimeZone=LOCAL&forceConnectionTimeZoneToSession=false`. `hibernate.jdbc.time_zone=UTC` nu ajută. Testele pe H2 nu folosesc acest URL.
3. **Calcul pur** (fără bază, fără Spring în calcul, câte un test unitar). `SimulationClock`: prima oră e 2026-06-01T00:00; următoarea = ultimul `recorded_at` + 1 h, deci aplicația repornită continuă de unde a rămas. `PvGenerator`: între orele 6 și 17, PV = vârf × sin(π × (oră − 6) / 12); în rest 0. `LoadGenerator`: bază pe interval (0–5: 0.6 kW · 6–16: 0.9 · 17–22: 1.8 · 23: 0.8) × (0.9 + 0.2 × *r*), cu *r* = `new Random(seed + houseId × 1000 + indexul orei de la start, adică orele scurse de la 2026-06-01T00:00).nextDouble()`. `PriceGenerator`: 0.5000 lei/kWh între 0–5, 1.2000 între 17–21, 0.8000 în rest (`BigDecimal`).
4. **Repository.** Decomentezi `TelemetryRepository`: `findByHouseIdOrderByRecordedAtDesc(Long, Pageable)` (apelat cu `PageRequest.of(0, 50)`), plus ultima oră înregistrată și rândurile unei ore. `HouseRepository`: casele cu `enabled = true`.
5. **SimulationService.tick()** (`simulation/service/commandService`, un singur `@Transactional`). Pentru fiecare casă activă: PV, consum, preț → `EnergyEngine.compute(pv, load, 0.0, 1.0)` → un `Telemetry` (SoC = cel curent al bateriei, `batteryPowerKw` 0.0, `gridPowerKw` = `netPowerKw`); un singur `saveAll`. Răspunde cu `SimulationTickResponse(tick, simulationTime, telemetryRows)`. O linie `log.info` per tick („Tick 13 la 2026-06-01T12:00: 10 case”).
6. **DemoDataSeeder** (`@Profile("demo")`, `ApplicationRunner`): dacă `houseRepository.count() > 0` nu face nimic; altfel creează cei 10 proprietari (tip `USER`, parola codată) și casele prin `createHouse` din F2, deci cu aceeași baterie.
7. **Citire.** Serviciu cu `getHouseTelemetry(houseId, email)` (ultimele 50) (accesul prin `checkHouseAccess`) și `getCommunityState()`: valorile ultimului tick, însumate. `simulationStatus` = `PAUSED` (tick-ul e manual; `RUNNING` ar însemna rulare automată, care nu e în plan); `currentTick` = orele scurse de la start + 1, sau 0.

8. **Intrare.** `SimulationController` (`simulation/controller`) și `TelemetryController` (folder nou, `telemetry/controller`), cu `SIMULATION_CONTROL`, `TELEMETRY_VIEW`, `COMMUNITY_VIEW`.
9. **Test.** Unitare pe generatoare, ceas și serviciu; integrare (H2, login real) pe controllere și seeder.

Endpoint	Permisiune	Cerere → răspuns
`POST /api/v1/simulation/tick`	`SIMULATION_CONTROL`	fără corp → **201** `{"tick":13,"simulationTime":"2026-06-01T12:00:00","telemetryRows":10}`
`GET /api/v1/houses/{houseId}/telemetry`	`TELEMETRY_VIEW` + proprietar	**200**, listă de cel mult 50, cele mai noi primele: `[{"id":121,"houseId":1,"recordedAt":"2026-06-01T12:00:00", "pvPowerKw":5.0,"loadPowerKw":0.9,"gridPowerKw":4.1, "batterySocPercent":50.0,"batteryPowerKw":0.0,"pricePerKwh":0.8000}, …]` Consumul „0.9” e un exemplu: cu sămânța 42 e între 0.81 și 0.99.
`GET /api/v1/community/state`	`COMMUNITY_VIEW`	**200** `{"totalHouses":10,"totalPvPowerKw":50.0,"totalLoadPowerKw":9.0, "totalGridPowerKw":41.0,"averageBatterySocPercent":50.0, "simulationStatus":"PAUSED","currentTick":13}` (`totalLoadPowerKw` ≈ 9, `totalGridPowerKw` ≈ 41: depind de sămânță.)

Erori: **403** la tick pentru oricine în afară de ADMIN, la telemetria altei case și la `/community/state` pentru `USER` · **404** `"message":"House not found"`.

Reguli

1. Aceeași sămânță, aceeași casă și aceeași oră → aceleași valori. Noaptea (înainte de 6 și de la 18) PV = 0; la 12:00 PV = vârful casei.
2. Un tick = o oră virtuală. Prima oră: 2026-06-01T00:00. Ora următoare: ultimul `recorded_at` + 1 h (rezistă la repornire).
3. Casa cu `enabled = false` nu primește rând. Până la agent, bateria stă `IDLE` în tick: `batteryPowerKw` 0, SoC neschimbat (comenzile manuale din F3 îl pot muta între tick-uri).
4. `gridPowerKw` = PV − consum + baterie: pozitiv = export, negativ = import.
5. Tick-ul și seeder-ul: doar `SIMULATION_CONTROL` (403), respectiv doar profilul `demo`. A doua pornire nu mai creează nimic.
6. Telemetria casei altcuiva → 403. Lista are cel mult 50 de rânduri, cele mai noi primele.

Teste de scris	Ce verifică
`PvGeneratorTests`	`night_isZero` (orele 0, 3, 5, 18, 23) · `noon_equalsPeak` (5.0 la 12) · `nineAm_isPeakTimesSin45` (3.5355, delta 0.0001)
`LoadGeneratorTests`	`sameSeedHourHouse_sameValue` · `differentHouse_differentValue` · `midnight_between_0_54_and_0_66`
`PriceGeneratorTests` · `SimulationClockTests`	0:00 → 0.5000, 12:00 → 0.8000, 18:00 → 1.2000 · prima oră = start; următoarea = ultima + 1 h
`SimulationServiceTests.tick_savesOneRowPerEnabledHouse` (Mockito)	`findByEnabledTrue()` stub-uit cu 2 case → `saveAll` cu 2 rânduri (filtrarea o face query-ul, nu serviciul); bateria rămâne `IDLE`
`SimulationControllerIntegrationTests` (H2, login real)	`tickAsAdmin_returns201_andWritesRows` · `tickAsUser_returns403` · `secondTick_isNextHour` · `telemetry_ownerSeesOwn_strangerGets403_newestFirst` · `communityState_operator200_user403`
`DemoDataSeederIntegrationTests` (profil `demo`)	10 case, 10 baterii; a doua pornire nu adaugă nimic

Gata când (bază goală: `docker compose down -v && up -d mysql-server`; admin ca în F1):

```
SPRING_PROFILES_ACTIVE=demo ADMIN_EMAIL=admin@solar.ro ADMIN_PASSWORD='Admin1234!' mvn spring-boot:run
ADMIN=$(tok admin@solar.ro 'Admin1234!')       # B, J și tok() din F1
curl -s -o /dev/null -XPOST $B/users -H "$J" -H "Authorization: Bearer $ADMIN" -d '{"firstName":"Mihai",
 "lastName":"Rus","email":"mihai@solar.ro","password":"parola123","age":34,"userType":"OPERATOR"}'
OPERATOR=$(tok mihai@solar.ro parola123)
for i in $(seq 1 10); do curl -s -o /dev/null -w '%{http_code} ' -XPOST $B/simulation/tick \
  -H "Authorization: Bearer $ADMIN"; done        → 201 201 201 201 201 201 201 201 201 201
M="docker compose exec -T mysql-server mysql -uroot -proot solarsyncbroker_db -e"
$M "select count(*), count(distinct recorded_at) from telemetry"        → 100   10
3 tick-uri în plus (13 în total):
$M "select pv_power_kw from telemetry where house_id=1 and recorded_at='2026-06-01 12:00:00'"
  → 5.0
$M "select sum(pv_power_kw) from telemetry where recorded_at='2026-06-01 12:00:00'"   → 50
$M "select sum(pv_power_kw) from telemetry where recorded_at='2026-06-01 00:00:00'"   → 0
$M "select round(sum(load_power_kw),6) from telemetry"   → notezi valoarea V
curl -s $B/houses/1/telemetry -H "Authorization: Bearer $(tok owner1@solar.ro 'Demo1234!')"
  → 200, 13 obiecte (grep -o '"id"' | wc -l) ; cu owner2 → 403
curl -s $B/community/state -H "Authorization: Bearer $OPERATOR"  → totalHouses 10, totalPvPowerKw 50.0, currentTick 13
down -v, aceeași pornire, 13 tick-uri, aceeași interogare pe load → V identic
mvn test → BUILD SUCCESS, Failures: 0, Errors: 0
```

Capcane

- **`LocalDateTime.now()` în loc de ceasul virtual.** Testele nu se mai repetă și două rulări nu dau aceleași valori. Toată logica de timp trece prin `SimulationClock`.
- **Un `Random` comun.** Dacă ținem un singur `Random` pentru toate casele, valorile unei case depind de ordinea în care sunt procesate celelalte. Fiecare valoare își face generatorul din sămânță + casă + oră.
- **Două cereri deodată.** Ora următoare se calculează din ultimul rând; două tick-uri simultane încearcă aceeași oră și unicul din bază o oprește pe a doua (eroare de integritate). Sunt cereri de administrator, făcute pe rând, dar testul tău de concurență să nu aștepte 201 de la amândouă.
- **Bateria citită în afara tranzacției.** `open-in-view` e oprit, deci `house.getBattery()` dă `LazyInitializationException` dacă nu ești în `@Transactional`. Tick-ul e o singură tranzacție: dacă pică o casă, nu rămâne un tick pe jumătate.

F5 Banii casei

proprietar · operator

Cu ce știi: da. `BigDecimal` e Java de bază, nu cere lecție. Nicio unealtă nouă.

Ce poate face utilizatorul: la fiecare tick, importul casei se înregistrează ca `BUY` și exportul ca `SELL`, la prețul orei; proprietarul își vede tranzacțiile casei, cele mai noi primele, fiecare cu valoarea ei; operatorul le vede pe cele ale comunității.

Scenariu (după F4, 13 tick-uri; casa 1, vârf 5 kW)

1. La tick-ul 1 (00:00) casa nu produce și consumă ~0.6 kW la 0.5000 lei/kWh: apare un rând `BUY` de 0.60 kWh, valoare 0.30 lei.
2. La tick-ul 13 (12:00) casa produce 5.0 kW și consumă ~0.9: exportă 4.1 kWh la 0.8000 → un rând `SELL` de 3.28 lei.
3. `owner1` cere `GET /api/v1/houses/1/financial-summary` și primește lista tranzacțiilor casei, cele mai noi primele. `owner2` pe aceeași casă → 403.
4. Operatorul cere `GET /api/v1/community/financial-summary`: aceeași formă, pe toate casele.

Ce ai deja

- `FinancialCommandServiceImpl.recordTransaction(houseId, type, amountKwh, unitPrice)` (kWh × preț, salvează) + 1 test. `FinancialQueryServiceImpl` cu `@Transactional(readOnly = true)` la clasă (corect: doar citește). `FinancialController` cu 2 endpoint-uri, `FinancialMapper`, `FinancialTransactionRepository.findByHouseId`.

Ce lipsește sau e stricat

1. **B7** · `recordTransaction` n-are niciun apelant (tabelul se umple doar de mână); lucrează cu `double` (`FinancialCommandServiceImpl.java:23`, deși `BigDecimal` e importat la `:8`) și pune `LocalDateTime.now()` la `:26`, nu ora tick-ului.
2. **M1** · `FinancialController.java:15`: `@RequestMapping("/houses")`, fără `/api/v1`; `:36`: `/houses/community/financial-summary` funcționează (rutarea merge), dar `community` stă pe locul unui `{id}` de casă și lipsește `/api/v1`; `:24` nu verifică proprietarul: oricine are `FINANCIAL_VIEW` vede orice casă. `FINANCIAL_VIEW` există în enum (`UserPermissions.java:14`), dar nimeni nu o primește din harta veche, iar `COMMUNITY_VIEW` lipsește din enum (intră în F1): de aici 403 cu token real.
3. `FinancialTransaction.java:10`: tabela `financial_transaction` (singular), câmpul `timestamp`, `houseId` ca `Long` fără cheie străină în bază, coloane nullable, sume `double`.
4. `FinancialControllerIntegrationTest`: 3 teste cu `@WithMockUser(authorities = …)` care își inventează permisiunile; trec și când endpoint-ul nu merge pentru nimeni logat.

1. **Bază.** `V5__financial.sql`: `financial_transaction` → `financial_transactions`; `timestamp` → `recorded_at`; toate coloanele `NOT NULL`; `transaction_type VARCHAR(10)`; `unit_price DECIMAL(10,4)`; `total_value DECIMAL(12,2)`; cheie străină pe `house_id`; unic pe `(house_id,`

`recorded_at`). Entitatea în același commit: `@Table(name = "financial_transactions")`, `recordedAt`, `BigDecimal unitPrice` și `totalValue`, `amountKwh` rămâne `double` (energie, nu bani).

2. **Service.** `recordTransaction(houseId, type, amountKwh, unitPrice, recordedAt)`: `totalValue = BigDecimal.valueOf(amountKwh) × unitPrice`, `setScale(2, RoundingMode.HALF_UP)`. `SimulationService.tick()` (F4) cheamă după `compute`: import > 0 → `BUY` cu `gridImportKwh`; export > 0 → `SELL` cu `gridExportKwh`; prețul e cel din telemetria aceluiași tick, ora e ora virtuală. Aceeași tranzacție ca telemetria.
3. **Citire.** `findByHouseIdOrderByRecordedAtDesc` în repository; `getAllTransactions` tot în ordine descrescătoare. `FinancialtransactionResponseToDto`: `timestamp` → `recordedAt`, `unitPrice` și `totalValue` devin `BigDecimal`; `FinancialMapper` se aliniază.
4. **Intrare.** `/api/v1/houses/{houseId}/financial-summary` (`FINANCIAL_VIEW`, apoi `checkHouseAccess` din F2) și `/api/v1/community/financial-summary` (`COMMUNITY_VIEW`).
5. **Test.** Cele 3 teste cu `@WithMockUser` devin teste cu login real; `grep -rn WithMockUser src/test` nu mai găsește nimic.

Endpoint	Permisiune	Răspuns

`GET /api/v1/houses/{houseId}/financial-summary`	`FINANCIAL_VIEW` + proprietar	**200**, listă, cele mai noi primele:
		`[{"id":3,"recordedAt":"2026-06-01T13:00:00","houseId":1,`
		`"transactionType":"SELL","amountKwh":3.2,"unitPrice":0.8000,"totalValue":2.56},`
		`{"id":2,"recordedAt":"2026-06-01T12:00:00","houseId":1,`
		`"transactionType":"SELL","amountKwh":4.1,"unitPrice":0.8000,"totalValue":3.28},`
		`{"id":1,"recordedAt":"2026-06-01T00:00:00","houseId":1,`
		`"transactionType":"BUY","amountKwh":0.62,"unitPrice":0.5000,"totalValue":0.31}]`
		Forma e cea reală (`unitPrice` cu 4 zecimale, din `DECIMAL(10,4)`; `totalValue` cu 2); cifrele sunt un exemplu cu 3 rânduri (4.1 × 0.8 = 3.28, 0.62 × 0.5 = 0.31).

`GET /api/v1/community/financial-summary`	`COMMUNITY_VIEW`	**200**, aceeași formă, rândurile tuturor caselor

Erori: **403** `"message":"House access denied"` (casa altcuiva) sau permisiune lipsă (`USER` pe `/community/…`) · **404** `"message":"House not found"`.

Reguli

1. `BUY` = importul tick-ului, `SELL` = exportul. Un tick cu net exact 0 nu scrie rând. Cel mult un rând pe casă și tick (unicul din bază): același tick procesat de două ori nu dublează banii.
2. Valoarea = kWh × preț, pe `BigDecimal`, rotunjită la 2 zecimale `HALF_UP`: 1.2345 kWh × 0.8000 = 0.98760 → **0.99**.
3. Prețul tranzacției e prețul din telemetria aceluiași tick, nu unul recalculat.
4. Cele mai noi primele, în ambele endpoint-uri.
5. Proprietarul vede doar casa lui (403 altfel); `/community/…` doar operatorul și administratorul.

Teste de scris	Ce verifică
`FinancialCommandServiceImplTest.recordTransaction_multipliesKwhByPrice` (Mockito)	0.62 kWh × 0.5000 → `totalValue` 0.31 (captor pe `save`)
`…roundsHalfUp` · `…threeTimesTenCentsIsThirtyCents`	1.2345 × 0.8000 → 0.99 · trei vânzări de 0.10 însumate dau exact 0.30 (cu `double` ar da 0.30000000000000004)
`SimulationServiceTests.tick_recordsBuyWhenImporting_sellWhenExporting_noneWhenZero`	motorul mockat întoarce import 0.6 → un BUY; export 4.1 → un SELL; net 0 → nimic; prețul e cel din telemetrie
`FinancialQueryServiceTests.getTransactionsByHouseId_usesNewestFirstQuery`	serviciul cheamă metoda cu ordine descrescătoare și întoarce rândurile ei
`FinancialControllerIntegrationTest` (H2, login real)	`owner_seesOwnSummary` · `stranger_gets403` · `operator_seesCommunity` · `user_onCommunity_gets403` · `afterTicks_valuesMatchTelemetry` (valoarea fiecărui rând = lgridl × preț, ±0.005)

Gata când (de la zero, cu aceleași variabile ca în F1):

```
docker compose down -v && docker compose up -d mysql-server
SPRING_PROFILES_ACTIVE=demo ADMIN_EMAIL=admin@solar.ro ADMIN_PASSWORD='Admin1234!' mvn spring-boot:run
ADMIN=$(tok admin@solar.ro 'Admin1234!')
curl -s -o /dev/null -XPOST $B/users -H "$J" -H "Authorization: Bearer $ADMIN" -d '{"firstName":"Mihai",
 "lastName":"Rus","email":"mihai@solar.ro","password":"parola123","age":34,"userType":"OPERATOR"}'
OPERATOR=$(tok mihai@solar.ro parola123)
for i in $(seq 1 10); do curl -s -o /dev/null -XPOST $B/simulation/tick -H "Authorization: Bearer $ADMIN"; done
$M "select count(*) from financial_transactions"                          → 100
$M "select transaction_type,count(*) from financial_transactions group by transaction_type"
  → BUY n1 și SELL n2, cu n1 + n2 = 100
$M "select count(*) from financial_transactions f join telemetry t
    on t.house_id=f.house_id and t.recorded_at=f.recorded_at
    where abs(f.total_value - abs(t.grid_power_kw) * t.price_per_kwh) > 0.0051"   → 0
curl -s $B/houses/1/financial-summary -H "Authorization: Bearer $(tok owner1@solar.ro 'Demo1234!')" \
  | grep -o '"id"' | wc -l        → 10 (un rând pe tick, casa 1); primul din listă are cel mai mare id
… cu owner2 → 403 ; $B/community/financial-summary cu $OPERATOR → 200 ; cu owner1 → 403
mvn test ; grep -rn WithMockUser src/test     → BUILD SUCCESS ; nimic
```

Pe EC2, cu token real: `financial-summary` → 200.

Capcane

- `new BigDecimal(0.1)`. Construiește valoarea binară a lui 0.1, nu 0.1. Folosești `BigDecimal.valueOf(0.1)` sau `new BigDecimal("0.1")`.
- `multiply` fără `setScale`. 1.2345 × 0.8000 are 5 zecimale; baza are 2, iar fără rotunjire explicită Hibernate trunchiază în loc să rotunjească. Rotunjești tu, `HALF_UP`.
- **Preț recalculat în loc de citit.** Dacă `recordTransaction` cheamă din nou generatorul, un test care schimbă prețul în telemetrie nu mai prinde nimic. Prețul se trece ca parametru, din același tick.

F6 Testele rulează pe MySQL real

tu · CI

Nou: Testcontainers + failsafe — lecția `spring-testcontainers` înainte (gata). Lecția e scrisă pe Boot 4.1; proiectul tău e pe Boot 3.5: diferențele sunt la pasul 1.

Ce poate face dezvoltatorul: rulezi `mvn -B verify` și testele de integrare merg pe un MySQL real, cu migrările V1–V5 aplicate de Flyway; un pull request cu un test roșu nu poate intra în `master`.

Scenariu

1. `mvn -B test` rulează doar testele unitare (Mockito), fără Docker, în câteva secunde.
2. `mvn -B verify`: Testcontainers pornește `mysql:8.4`, Flyway aplică cele 5 migrări, apoi failsafe rulează toate testele `*IT` pe el.
3. Faci un test roșu intenționat într-un PR: check-ul „test” din Actions e roșu și GitHub blochează merge-ul.

Ce ai deja

- Testele din F1–F5: unitare Mockito și `@SpringBootTest` + MockMvc pe H2 (profilul `test`), cu login real; încă 16 teste în `HouseCommandServiceIT`, `HouseQueryServiceIT` și `BatteryCommandServiceIT`, rulate pe nume.
- Dependențele `spring-boot-testcontainers`, `org.testcontainers:junit-jupiter` și `org.testcontainers:mysql` sunt deja în `pom.xml`, neîntrebuințate. Jobul `test` din `build-push.yaml`.

Ce lipsește sau e stricat

1. **B9** · `pom.xml` n-are `maven-failsafe-plugin`: testele `*IT` nu rulează la `mvn test` și nici în CI; `com.h2database:h2` e încă în pom.
2. Testele de integrare nu văd migrările: pe H2 (`application-test.yml`, Flyway oprit, `create-drop`) tabelele le face Hibernate, deci o eroare în V1–V5 trece neobservată până la pornirea pe MySQL.
3. `build-push.yaml` rulează `mvn -B test` doar la push pe `master`; nu există workflow pe `pull_request` și nu e check obligatoriu.

1. **Failsafe și dependențe.** `pom.xml`: adaugi `maven-failsafe-plugin` în `build/plugins`, fără configurație (părintele Spring Boot îl pornește pe `integration-test` și `verify`); ștergi `com.h2database:h2`. Dependențele Testcontainers rămân cele din pom: lecția are artefactele noi din Testcontainers 2.x, iar Boot 3.5 folosește 1.x.
2. **Containerul.** În `src/test/java`: `TestcontainersConfiguration` (`@TestConfiguration(proxyBeanMethods = false)`, cu un `@Bean @ServiceConnection` de tip `MySQLContainer<?>` pentru `"mysql:8.4"`) și o clasă de bază `AbstractIT` cu `@SpringBootTest`, `@AutoConfigureMockMvc`, `@Import(TestcontainersConfiguration.class)`. **Atenție la import:** la tine e `org.testcontainers.containers.MySQLContainer`, nu `org.testcontainers.mysql.MySQLContainer` din lecție.
3. **Mutarea testelor.** Fiecare clasă cu `@SpringBootTest` devine `*IT extends AbstractIT`: `AuthControllerIntegrationTests`, `UserControllerIntegrationTests`, `BatteryControllerIntegrationTests`, `FinancialControllerIntegrationTest`,

`HouseControllerIntegrationTests`, `SimulationControllerIntegrationTests`, `DemoDataSeederIntegrationTests`, **și** `SolarSyncBrokerApiApplicationTests`, `AdminBootstrapTests` (nume `*Tests`, dar pornesc aplicația). Cele 3 clase `…ServiceIT` extind și ele `AbstractIT`. Șterge `@ActiveProfiles("test")` și `src/test/resources/application-test.yml`. Doar testele fără Spring (Mockito) rămân `*Test(s)`, la `mvn test`.

4. **Prima rulare.** `mvn -B verify` înainte de orice altă schimbare: ce a mers pe H2 poate pica pe MySQL (ordinea ștergerii în `@BeforeEach`, valori de dată, default-uri). Le repari pe loc; de aici, migrările V1–V5 sunt aplicate și verificate în fiecare rulare.
5. **Verificarea migrărilor.** `SchemaIT`: numărul rândurilor cu `success = 1` din `flyway_schema_history` e egal cu numărul fișierelor `V*.sql` din `db/migration` (le numeri în test: nu pui „5" în cod, ca să nu pice la următoarea migrare).
6. **CI.** `build-push.yaml`, jobul `test`: `mvn -B verify` (runner-ul GitHub are Docker). Fișier nou `.github/workflows/ci.yaml`, pe `pull_request`, același `verify`. În GitHub, *Settings → Branches → Require status checks*: pui check-ul obligatoriu (de mână, în browser). De aici, fiecare feature merge pe branch cu PR.

Testele existente pe care le rupe mutarea pe MySQL

- `SolarSyncBrokerApiApplicationTests` și `AdminBootstrapTests`: rămase în surefire, pornesc aplicația fără MySQL și `mvn -B test` pică cu `Communications link failure` după ce H2 iese.
- Orice clasă cu `@ActiveProfiles("test")` sau care citește `application-test.yml`: profilul dispare.
- Curățarea din `@BeforeEach`: dacă vreo clasă șterge doar baterii, case și utilizatori, cu chei străine reale pică pe `command_executions`; ștergerea trebuie să fie cea din F2, în ordinea cheilor străine.

Teste de scris	Ce verifică
`SchemaIT.flywayAppliedAllMigrations` (MySQL)	numărul rândurilor din `flyway_schema_history` cu `success = 1` = numărul fișierelor `V*.sql`; `validate` a trecut la pornire
testele existente, mutate pe `AbstractIT`	aceleași aserțiuni, acum pe MySQL real; niciuna nu se șterge sau se dezactivează ca să treacă

Gata când:

```
mvn -B test      → doar unitarele; niciun test *IT nu rulează
mvn -B verify    → mysql:8.4 pornit de Testcontainers, "Successfully applied 5 migrations",
                   failsafe: Failures: 0, Errors: 0, Skipped: 0 → BUILD SUCCESS
grep -rn 'ActiveProfiles("test")\|h2database' src pom.xml     → nimic
PR cu un test făcut roșu intenționat → check-ul „test" e roșu și merge-ul e blocat
```

Capcane

- **Docker oprit.** `Could not find a valid Docker environment`: pornești Docker Desktop. Pornește un container per context Spring distinct: `AdminBootstrapIT` (cu `properties`) și `DemoDataSeederIT` (profilul `demo`) au al lor, restul claselor `*IT` împart unul, deci **datele rămân între clase**: fiecare clasă curăță în `@BeforeEach`.
- **Importurile din lecție.** Lecția e pe Testcontainers 2.x (`testcontainers-mysql`, `org.testcontainers.mysql.MySQLContainer`); tu ai 1.x. Dacă IDE-ul nu găsește clasa, ai importul greșit, nu dependența.
- **Numele clasei.** `*IntegrationTests` nu e `*IT`: failsafe nu-l găsește și `mvn verify` iese „verde" fără el. După redenumire verifici că `Tests run:` din failsafe e mai mare ca 0.

F7 Logarea zilei simulate

administrator · cine citește logurile

nou logare structurată (MDC, JSON) · lecția `spring-logare-structurata` înainte (de făcut) · **Pornești de la:** `SimulationService.tick()` din F4, cu `log.info` simplu

Fiecare linie de log din timpul unui tick poartă id-ul tick-ului și al casei; pe EC2 logurile ies ca JSON, ca să le poți filtra.

1. **Id-ul fluxului.** La începutul lui `tick()`, `tickId` (numărul tick-ului) intră în MDC și iese în `finally`; în bucla pe case, `houseId` la fel.
2. **Niveluri.** O linie `INFO` per tick (câte case, cât a durat), `DEBUG` per casă; pachetul `simulation` pe `DEBUG` în profilul de dezvoltare.
3. **Format.** Local, pattern text cu `%X{tickId}`. În profilul `ec2`: `logging.structured.format.console` (Spring Boot 3.4+, ai 3.5), deci câmpurile MDC apar ca JSON pe fiecare linie. Profilul se pornește din compose-ul platformei.
4. **Test.** `OutputCaptureExtension`: după un tick, fiecare linie conține `tickId`; după tick, MDC-ul e gol.

Gata când: 3 tick-uri, apoi `docker compose logs solar-sync-app | grep '"tickId":"2"'` → 11 linii (1 `INFO` + 10 `DEBUG`) · în logurile de pe EC2 filtrezi un singur tick după `tickId`.

F8 Starea rețelei

operator · administrator

Nou: Micrometer + Prometheus + Grafana — lecția `devops/metrici-prometheus-grafana` înainte (de făcut).

La fiecare tick se calculează încărcarea transformatorului comun și se salvează. Operatorul vede starea; în Grafana vede comunitatea live.

Pornești de la: `GridProperties` (50 kW, marjă 10%), tick-ul din F4 și `ComunityStateResponse` · nou Micrometer + Prometheus + Grafana

1. **Bază.** `V6__grid_snapshots.sql`.
2. **Calcul pur.** `GridGuard`: din importul și exportul total + `GridProperties` → stare și kW în plus. Teste pe cele trei praguri.
3. **Service + intrare.** Tick-ul salvează un rând. `GET /api/v1/grid/state` (`GRID_VIEW`).
4. **Metrici.** Actuator + `micrometer-registry-prometheus`. Gauge-uri: import și export total, starea rețelei, SoC mediu. Contoare: comenzi pe status, tick-uri. Timer: durata tick-ului. `/actuator/prometheus` se vede doar din rețeaua compose.
5. **Ops.** Prometheus și Grafana în compose; dashboard-ul (JSON) și sursa de date în `solar-sync-broker-ops/grafana/`, încărcate automat la pornire.

Reguli

- Limita = 50 kW × (1 – 10%) = 45 kW. Import și export separat: ≤ 45 `OK`, până la 50 `WARNING`, peste `CRITICAL`, cu kW-ii în plus.

Gata când: comunitatea împinsă artificial peste 50 kW → `/grid/state` = `CRITICAL` și panoul din Grafana se face roșu · după 24 de tick-uri, graficul SoC-ului mediu arată ziua.

II. MCP — piesa centrală

Comunitatea devine ceva ce poate folosi un agent: întâi doar citind, apoi propunând. Primul microserviciu apare aici, firesc.

F9 Comunitatea văzută de un agent

orice client MCP

Nou: server MCP (Spring AI) — lecția `java/spring-ai/mcp-server` înainte (de făcut).

Un client MCP (MCP Inspector, Claude Desktop, mai târziu brokerul tău) se conectează cu un token și întreabă de case, baterii, telemetrie, prețuri, rețea și bani. Vede exact cât vede utilizatorul tokenului.

Pornești de la: query service-urile din F2–F5 și F8 · **nou** server MCP (Spring AI 1.1, `spring-ai-bom` e deja în `pom.xml`)

1. **Dependență.** Starter-ul de server MCP pe WebMVC, transport Streamable HTTP.
2. **Tool-uri, în pachetul `mcp/`.** `listHouses`, `getHouse`, `getBattery`, `getTelemetry(houseId, last ≤ 50)`, `getCommunityState`, `getGridState`, `getPriceHistory(hours ≤ 24)`, `getFinancialSummary(houseId)`.
3. **Regula tool-urilor.** Un tool = un apel la un service existent. Fără repository, fără logică în tool. Descrierea tool-ului și a fiecărui parametru e scrisă pentru cine o citește: un model.
4. **Permisiuni.** Endpoint-ul MCP stă în spatele aceluiași filtru JWT. Verificarea proprietarului rămâne în service, deci e aceeași prin REST și prin MCP.
5. **Log / metrică.** Fiecare apel: numele tool-ului și utilizatorul, într-o linie de log și un contor per tool.

Gata când: MCP Inspector cu token de OPERATOR listează cele 8 tool-uri, iar `getGridState` dă același rezultat ca REST · cu tokenul unui proprietar, `getTelemetry` pe casa altcuiva → eroare de acces · fără token → 401.

F10 Propun → aprob → execut

agent · operator

Nou: tool-uri MCP de scriere — lecția `java/spring-ai/mcp-server, partea a doua` înainte (de făcut).

Un agent propune un plan: o decizie per casă (`CHARGE` / `DISCHARGE` / `IDLE`, kW, motiv). Ce se întâmplă cu planul decide `AgentMode`. Orice execuție trece prin poarta din F3.

Pornești de la: `AgentMode`, `AgentProperties`, `executeCommand` (F3), `BatteryCommandValidator`, `CommandExecution` · **nou** tool-uri MCP de scriere

1. **Bază.** `V7__agent.sql`: `agent_runs`, `agent_decisions`, `command_executions.agent_decision_id`. Contul `AGENT` îl creează administratorul, prin `POST /users`.
2. **Service.** Propunerea salvează run-ul și fiecare decizie, trecută prin validator **fără să atingă bateria**: `valid` + `rejection_reason`. Aprobarea cheamă `executeCommand` din F3 pentru fiecare decizie validă, cu `executed_by` = agentul + cine a aprobat.
3. **Modurile.** `DISABLED`: propunerea e refuzată · `ADVISORY`: doar se salvează · `SUPERVISED`: așteaptă aprobarea unui om · `AUTONOMOUS`: se execută imediat, tot prin poartă.
4. **Intrare.** Tool-uri MCP `proposePlan(decisions)` (`AGENT_PROPOSE`) și `getRun(id)`. REST: `GET /api/v1/agent/runs`, `GET /api/v1/agent/runs/{id}` (`AGENT_VIEW`), `POST /api/v1/agent/runs/{id}/approve` și `…/reject` (`AGENT_APPROVE`; respingerea nu schimbă nicio baterie, doar închide rularea).
5. **Test.** Contul `AGENT` propune; operatorul aprobă; agentul care încearcă să aprobe singur → 403.

Gata când: din MCP Inspector, cu token `AGENT`, în `SUPERVISED`: un plan cu o decizie de 2 kW și una de 90 kW → după aprobare: o comandă `SUCCESS` cu `agent_decision_id`, iar decizia de 90 kW rămâne neexecutată, cu motivul.

F11 Brokerul pe reguli — primul microserviciu

agent

Nou: client MCP, a doua aplicație — lecția `java/spring-ai/mcp-client` înainte (de făcut).

O aplicație separată, `solar-sync-broker-agent`, care la fiecare oră virtuală citește comunitatea prin MCP, aplică regulile tale și propune planul. Nu are bază de date și nu cunoaște API-ul REST, doar tool-urile.

Pornești de la: regulile din `Descrierea Simplă a Proiectului.pdf`, `AgentProperties.planningIntervalSeconds`, contul `AGENT` · **nou** client MCP, a doua aplicație

1. **Proiect.** Folderul `solar-sync-broker-agent/`: Boot 3.5, starter-ul de client MCP. La pornire se loghează ca `AGENT` (credențiale din `.env`) și trimite tokenul la serverul MCP.
2. **Calcul pur.** Interfața `AgentPlanner` și `RuleBasedPlanner`, câte o metodă pe regulă, cu teste unitare pe fiecare (în lucrare, fiecare devine un paragraf).
3. **Bucla.** La interval (`@Scheduled` cu `@EnableScheduling`, o adnotare explicată în lecția client MCP): tool-uri de citire → planner → `proposePlan`. Fiecare rulare are un `runId` în MDC.
4. **Ops.** Dockerfile propriu, serviciu nou în compose, job nou în CI, filtrat pe `paths`.

Reguli

- Pragul = media prețurilor din ultimele 24 de ore virtuale. Preț < 0,75 × prag și SoC < 90 → `CHARGE` la puterea maximă. Preț > 1,4 × prag și SoC > 20 → `DISCHARGE` la puterea maximă. Altfel `IDLE`.
- Rețeaua `WARNING` / `CRITICAL` are prioritate: descărcările se reduc până când exportul total intră sub 45 kW.

Gata când: pe EC2, cu ambele aplicații pornite: după un tick, în `/agent/runs` apare un plan · aprobat în `SUPERVISED` → SoC-urile se schimbă · agentul oprit → API-ul merge mai departe.

F12 Brokerul cu AI

agent

Nou: Spring AI `ChatClient` (Claude prin API) — lecția `java/spring-ai/chat-client` înainte (de făcut).

Același agent primește un al doilea planner, cu un model AI. Alegi din configurare care rulează. Rezultatul: comparația reguli vs. AI, pe aceeași zi simulată, pentru lucrare.

Pornești de la: `AgentPlanner` și clientul MCP din F11 · **nou** Spring AI `ChatClient`

1. **Planner.** `LlmPlanner`: `ChatClient` cu tool-urile MCP din F9 (modelul își cere singur datele) și ieșire structurată → lista de decizii. Prompt-ul stă într-un fișier din `resources`, cu limitele și scopurile comunității.
2. **Siguranță.** Timeout. Dacă modelul nu răspunde sau răspunde greșit, rulează `RuleBasedPlanner` și se scrie un `WARN`. Modelul e Claude, prin API-ul Anthropic: `spring-ai-starter-model-anthropic`, `spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}`, `spring.ai.anthropic.chat.options.model=${ANTHROPIC_MODEL:claude-opus-5-5}`; cheia stă în `.env` local și ca secret în Actions pentru EC2, niciodată în repo.
3. **Metrici.** Tokenii și latența apelurilor ajung în Prometheus, lângă celelalte.
4. **Comparația.** Aceeași sămânță, 24 de tick-uri, fiecare planner pe rând: profit, minute în `CRITICAL`, % decizii respinse de poartă, latență și cost.

Gata când: tabelul comparației e completat din bază și din Grafana · fără cheie API, agentul trece pe reguli și o spune în log.

III. Evenimente și servicii

Comunitatea trece de la „cine cheamă pe cine” la „cine ascultă ce”. La final ai tot drumul în aceeași aplicație distribuită: servicii, AI, evenimente și observabilitate, legate între ele.

F13 Tick-ul anunță, ceilalți reacționează

> **Nou: Kafka** — lecția `java/spring/spring-kafka` înainte (de făcut).

Pornești de la: tick-ul din F4, banii din F5, rețeaua din F8, agentul din F11 · **nou** Kafka

1. **Ops.** Kafka în compose, un singur nod (KRaft), în profilul `messaging`; topicurile le creează un container `kafka-init`, nu codul. **Kafka UI** (imaginea `provectuslabs/kafka-ui`) în același profil, ca să vezi topicurile și mesajele; în F16 îl pune oauth2-proxy în spate. Deploy-ul pe EC2 pornește doar profilurile care încap acolo.
2. **Evenimente.** După commit, tick-ul publică `telemetry.recorded` (per casă, cu `tickId`); rețeaua publică `grid.state`.
3. **Ascultători, tot în monolit.** Banii și rețeaua devin `@KafkaListener` în loc de apeluri directe din tick. Agentul pornește la `grid.state`, nu la timer.
4. **Idempotență.** Același eveniment primit de două ori nu dublează nimic (cheie unică pe casă + oră).

> **Gata când:** tick → tranzacțiile apar prin listener · același eveniment trimis de două ori → o singură tranzacție · listener oprit și repornit → recuperează ce a pierdut.

F14 Simulatorul pleacă din monolit

> **Nou: topicurile ca singur contract între servicii** — lecția `java/microservicii/al-doilea-serviciu` înainte (de făcut).

Pornești de la: generatoarele și `SimulationClock` din F4 · **nou** topicurile ca singur contract între servicii

1. `solar-sync-broker-simulator`: doar generatoarele și ceasul. Fără bază. Publică măsurătorile (PV, consum, preț) per casă și oră.
2. API-ul le consumă: aplică bateria, `EnergyEngine`, salvează telemetria și publică mai departe, ca în F13.

> **Gata când:** API-ul fără simulator merge, doar fără tick-uri · cu aceeași sămânță, telemetria e identică cu cea din F4.

F15 Urmăresc un tick prin tot sistemul

> **Nou: OpenTelemetry + Tempo + ELK** — lecția `devops/tracing-otel-tempo-elk` înainte (de făcut).

Pornești de la: logurile din F7 și metricile din F8 · **nou** OpenTelemetry + Tempo + ELK (Elasticsearch, Kibana, Filebeat)

1. **Trace-uri.** OpenTelemetry în toate cele trei aplicații, propagat prin headerele Kafka și prin apelurile MCP, trimis prin OTLP în Tempo; în Grafana vezi drumul unei cereri.
2. **Loguri.** Filebeat citește logurile JSON ale containerelor și le trimite în Elasticsearch; în Kibana cauți după `tickId` sau `traceId`. `traceId` intră în MDC, deci apare pe fiecare linie.
3. **Ops.** Profilurile `obs` (Tempo) și `logs` (Elasticsearch, Kibana, Filebeat). Elasticsearch cere 1–2 GB: local sau pe instanța de demo.

> **Gata când:** cauți un `tickId` în Kibana, iei `traceId`-ul și în Tempo vezi un singur drum: simulator → Kafka → API → rețea → agent → MCP → aprobare → comandă.

IV. Platforma, ca la cars

Ultimele două piese ale unei platforme reale: identitatea scoasă din aplicație și o singură poartă de intrare. Tot pe Docker.

F16 Cine ești, spus de Keycloak

toți · agentul · administratorul

Nou: Keycloak (OAuth2 / OIDC) — lecția `java/spring/spring-security-keycloak` înainte (de făcut).

Conturile și parolele nu mai stau în API. Keycloak emite tokenurile: pentru oameni (login), pentru agent (fără parolă) și pentru UI-urile de administrare.

Pornești de la: matricea de permisiuni din F1, `UserType`, `SecurityConfiguration`, `JWTTokenProvider`, contul `AGENT` din F10–F11 · **nou** Keycloak (OAuth2 / OIDC)

1. **Ops.** Keycloak și Postgres-ul lui în compose, profilul `auth`. Pornește cu `--import-realm`, din `realm-solar-sync.json` pus în `solar-sync-broker-ops/keycloak/`. Parola de admin și secretul clientului agentului: doar în `.env`.
2. **Realm.** Roluri `USER`, `OPERATOR`, `ADMIN`, `AGENT`. Clienți: `solar-sync-api`, `solar-sync-agent` (client credentials, cu rolul `AGENT`), `oauth2-proxy` și, pentru U4, `solar-sync-ui` (public, PKCE).
3. **Bază.** `V8__users_keycloak.sql`: `password` dispare, apare `keycloak_id`. Utilizatorul local rămâne, pentru că o casă are proprietar: la prima cerere cu un token nou, API-ul îl creează după `sub` și email.
4. **API.** Devine OAuth2 resource server pe issuer-ul realm-ului. Rolurile din token trec prin aceeași hartă rol → permisiuni din F1. `/auth/register`, `/auth/login` și `JWTTokenProvider` ies.
5. **Agentul.** Își ia tokenul prin client credentials, nu prin login cu parolă.
6. **UI-uri.** oauth2-proxy în fața Grafana, Kibana și Kafka UI: login prin Keycloak, doar grupul `/admins`.
7. **Test.** Testele cu login real iau tokenuri de la un Keycloak din Testcontainers, pornit cu același realm.
8. **Împreună cu U4.** Din momentul în care `/auth/login` iese, UI-ul se loghează prin Keycloak: U4 se face imediat după F16, înainte de F17.

Reguli

- Niciun rol nu se mai alege la înregistrare: contul de proprietar îl face Keycloak, rolurile le dă administratorul în realm.
- Matricea de permisiuni nu se schimbă; se schimbă doar cine emite tokenul.

Gata când: cu token din Keycloak, proprietarul → casa lui 200, casa altuia 403 · agentul, cu client credentials → `proposePlan` merge, aprobarea → 403 · vechiul `/auth/login` → 404 · Grafana fără login → pagina Keycloak; un utilizator din afara `/admins` → 403.

F17 O singură poartă: Kong

toți

Nou: Kong (API gateway) + rețele Docker — lecția `devops/kong-gateway` înainte (de făcut).

Aplicațiile nu mai sunt expuse direct. Tot traficul spre ele, inclusiv al agentului, trece printr-un API gateway care verifică tokenul, limitează abuzul și pune un id pe fiecare cerere.

Pornești de la: compose-ul din `solar-sync-broker-ops/`, tokenurile din F16, MDC-ul din F7 · **nou** Kong (API gateway) + rețele Docker

1. **Ops.** Kong DB-less în compose, cu `kong.yml` declarativ în `solar-sync-broker-ops/kong/`. Rute: `/api/v1` și `/mcp` → API. În fața lui, nginx doar pentru TLS (Let's Encrypt).
2. **Plugin-uri.** `jwt` cu cheia publică a realm-ului din F16 · `rate-limiting` · `correlation-id` (header-ul ajunge în MDC și în trace) · `prometheus` (metricile lui Kong în Grafana).
3. **Rețele Docker.** `edge`: nginx, Kong, oauth2-proxy · `backend`: aplicațiile, Kafka, MySQL, Keycloak. Doar Kong și oauth2-proxy sunt în amândouă. API-ul, agentul și simulatorul nu mai au `ports:`.
4. **Agentul.** Apelează MCP prin Kong, nu direct.
5. **UI-ul.** `/api` din containerul `ui` merge spre Kong, nu spre API.

Reguli

- Pe EC2 sunt deschise doar 80 și 443 (și 22, doar de la IP-ul tău). Orice alt port publicat e o greșeală.
- Kong verifică tokenul, dar permisiunile rămân în API: poarta nu ia decizii de domeniu.

Gata când: portul API-ului, încercat direct pe EC2 → nu răspunde · prin Kong fără token → 401 de la Kong, cu token → 200 · peste limita de cereri pe minut → 429 · id-ul de corelare din răspuns îl găsești în Kibana și în Tempo · în UI, toate cererile spre `/api` au header-ul de corelare pus de Kong.

V. Interfața — Angular (solar-sync-broker-client)

Se construiește în paralel cu backend-ul: fiecare U pornește doar după feature-ul de care are nevoie. Pornești de la ce ai făcut în `sebastian-angular-initiere` (șabloane, signal-uri, input-uri, condiții, liste).

U1 Casa mea, din browser

proprietar · după F3

Proprietarul se loghează, își vede casa și bateria, trimite o comandă și vede istoricul comenzilor.

Pornești de la: demo-urile din `sebastian-angular-initiere`, contractul din `openapi.yaml`, endpoint-urile din F1–F3 · **nou** Angular cu rutare, HttpClient și interceptor

1. **Proiect.** Aplicația Angular în `solar-sync-broker-client/`, componente standalone și signal-uri, ca în demo-uri. Un serviciu per resursă (`auth`, `houses`, `battery`), cu tipurile din contract.
2. **Login.** Formular → `/auth/login`; tokenul ține cât sesiunea. Un interceptor pune `Authorization: Bearer` pe fiecare cerere. 401 → înapoi la login.
3. **Pagini.** Casa mea (bateria: SoC, puteri maxime) · comandă (`CHARGE` / `DISCHARGE` / `IDLE` + kW) · istoricul, cu statusul și motivul la `REJECTED`. Erorile 403 și 404 apar ca mesaje, din `ApiErrorResponse`.
4. **Ops.** Dockerfile în două etape (build cu Node → fișiere statice servite de nginx), serviciul `ui` în compose (profilul `core`); `/api` merge spre API prin nginx-ul din containerul `ui`. Job `ui` în Actions, pornit doar de `solar-sync-broker-client/**`.
5. **Test.** Un test end-to-end (Playwright): login → comandă → apare în istoric.

> **Gata când:** pe EC2, proprietarul se loghează, trimite `CHARGE` 2 kW pe SoC 50 și vede SoC 69 și comanda `SUCCESS` în istoric · `CHARGE` 90 kW → `REJECTED` cu motivul · un străin care deschide casa → mesajul de acces interzis, nu o pagină goală.

U2 Comunitatea, live

operator · administrator · după F8

Operatorul vede starea comunității și a rețelei și telemetria unei case. Administratorul rulează un tick din pagină.

Pornești de la: U1, `/community/state`, `/houses/{id}/telemetry`, `/grid/state`, `/simulation/tick` · **nou** grafic (o bibliotecă de grafice) + reîmprospătare periodică

1. **Pagini.** Comunitatea: PV total, consum total, SoC mediu, starea rețelei (`OK` / `WARNING` / `CRITICAL`, colorată) · o casă: graficul ultimelor 50 de măsurători.
2. **Reîmprospătare.** La fiecare 10 s, oprită când pagina nu e vizibilă. Fără WebSocket.
3. **Roluri.** Meniul arată doar ce permite tokenul: proprietarul nu vede pagina comunității, butonul de tick apare doar la ADMIN.

> **Gata când:** ADMIN apasă tick → în cel mult 10 s, graficul casei are un punct nou și starea rețelei se actualizează, fără reîncărcarea paginii · proprietarul nu vede meniul comunității.

U3 Aprob planul agentului

operator · după F10 (F11–F12 pentru date reale)

Operatorul vede planurile propuse de agent, decizie cu decizie, și le aprobă sau le respinge.

Pornești de la: U2, `/agent/runs`, `/agent/runs/{id}/approve` și `/reject`

1. **Pagini.** Lista planurilor `PROPOSED` (cine, când, cu ce planner) · un plan: decizie per casă, `valid`, motivul, `rejection_reason` · Aprobă / Respinge.
2. **După aprobare.** Comenzile apar în istoricul fiecărei case (U1), cu agentul și cine a aprobat.

> **Gata când:** agentul propune → planul apare în listă în cel mult 10 s → aprobat din pagină → SoC-urile din U2 se schimbă · decizia de 90 kW e afișată ca invalidă, cu motivul, și nu se execută.

U4 Login prin Keycloak

toți · imediat după F16, înainte de F17

Formularul propriu de login dispare: utilizatorul se loghează pe pagina Keycloak. Se face odată cu F16, ca UI-ul să nu rămână nicio zi fără login.

Pornești de la: U1–U3, realm-ul din F16 · **nou** OIDC: authorization code + PKCE

1. **Realm.** Un client public `solar-sync-ui`, cu redirect-urile UI-ului, fără secret.
2. **UI.** O bibliotecă OIDC pentru Angular: login și logout prin Keycloak, tokenul reînnoit singur; interceptorul din U1 rămâne, doar tokenul vine din altă parte.

> **Gata când:** „Login” duce pe pagina Keycloak și înapoi în UI · un proprietar și un operator văd meniuri diferite · logout închide sesiunea și în Keycloak · vechiul formular de login nu mai există.

Toate cele 17 feature-uri și cele 4 de interfață fac parte din proiect. F-urile merg în ordine; fiecare U pornește doar după feature-ul scris la el, iar U4 se face între F16 și F17. Dacă susținerea vine înainte de final, în lucrare intră ce merge pe EC2 la data aceea.
Rămâi pe Spring Boot 3.5 și pe Spring AI 1.1 până la susținere (Spring AI 2.x cere Boot 4). Exemplele din proiectele pe Boot 4 nu se copiază 1:1.
Doar Docker, cu profiluri compose (`core`, `messaging`, `obs`, `logs`, `auth`, `edge`). Pe EC2-ul obișnuit: `core` + `obs`, apoi `auth` + `edge`. Kafka, simulatorul și Elasticsearch cer o instanță mai mare: local sau pe instanța de demo pentru susținere.
Secretele (`JWT_SECRET_KEY` până la F16, parolele, secretul clientului agentului, cheia API a modelului) stau doar în `.env`, pe EC2 și local. Niciodată în repo, niciodată în compose.